One Mehan Gardens, Cecilia Muñoz St., Ermita, Manila, 1000 Metro Manila, Philippines.

TOOTHWISE: TEETH CAVITY DETECTION WITH RECOMMENDATION SYSTEM USING YOLOv12 FOR CODISMILE DENTAL CLINIC

A Capstone Project Documentation
Presented to
The Department of Information System
Universidad De Manila

In Partial Fulfillment
of the Requirements for the Degree
BACHELOR OF SCIENCE IN INFORMATION SYSTEM

Brayan Axel L. Tubillas
Keith Ashly V. Sales
Reizen S. Arreola
Robin C. Ponce
Rizelle Carla N. Magpantay

Prof. Francis Arlando L. Atienza

April 2026

APPROVAL SHEET

In partial fulfillment of the requirements for the degree **BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY,** this Capstone project entitled **"TOOTHWISE: TEETH DETECTION WITH RECOMMENDATION SYSTEM USING YOLOv12 FOR CODISMILE CLINIC"**, has been prepared and submitted by **Arreola, Reizen S., Magpantay, Rizelle Carla N., Sales, Keith Ashly V., Tubillas, Brayan Axel L.** who are hereby recommended for project presentation.

Capstone Project Mentor

Approved by the Committee for Project presentation with a notation of _____ on April 15, 2026.

Chairperson, Project Presentation Committee

Panel Member *Panel Member*

Accepted and approved in partial fulfillment of the requirements for the degree **BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY**.

COLLEGE DEAN

Date: April 15, 2026

TABLE OF CONTENTS

Title Page ...……...……..

Approval Sheet…… ii

Dedication ……

Acknowledgement ……. iv

Executive Summary……. v

List of Tables…… vi

List of Figures…….…..... vii

List of Appendices…….. viii

Chapter 1 Capstone Project Background

Introduction

Project Background and Context

Project Purpose and Description

Main Objective

Specific Objectives

Project/System Description

Scope and Delimitations of the Project

Definition of Terms

Chapter 2 Review of Related Literature and Systems

Chapter 3 Methodology

3.1 Development Methodology

3.2 Project Planning

3.3 Roles and Responsibilities of the Development Team

3.4 Requirements Analysis

3.5 System Design (Use Case, ERD, DFD, Gantt Chart)

3.6 System Development

3.7 System Testing

3.8 Full Current System Screenshots

3.9 Technical Specifications

3.9.1 Software Specifications

3.9.2 Hardware Specifications

3.9.3 System Architecture

3.9.4 Database Technical Specifications

3.9.5 AI/ML Technical Specifications

3.9.6 Risk Assessment Technical Specifications

3.9.7 Recommendation Engine Technical Specifications

3.9.8 Security Technical Specifications

3.9.9 Network and Deployment Specifications

Chapter 5 Summary and Recommendation

Summary

Recommendations

References

Appendices …….

Evaluation Tools or Test Documents (for testing procedures)

Sample input / output / reports (Index)

Data / Process / Information Flow (ILLUSTRATIONS and DIAGRAMS. Ex: Process Flowchart

System Flowchart

Context Diagram

Data Flow Diagram (Exploded Process)

Site Map (if Web-Based)

VTOC (if Lan-Based)

Circuits Diagram (if Hardware)

Use Case or Activity Diagram

Network Diagram

Data Base Dictionary

Screen Layout

Developmental Cost

Picture showcasing the data gathering, investigation done (e.g floor plan, layout, building, etc…)

Certification from the Editor

Relevant Source Code (use two columns, font size: 10, and single spacing)

User Guide

One-page Curriculum Vitae per team member

List of Tables

Table No. Table Title

1.1 ISO/IEC 25010 Software Product Quality Criteria

3.1 Sources of Requirements and Technical Data

3.2 Roles and Responsibilities of the Development Team

3.3 Functional Requirements

3.4 Non-Functional Requirements

3.5 ToothWise Database Fact Table

3.6 Data Flow Dictionary for Level 0 Context Diagram

3.7 Capstone Project Work Breakdown Structure and Timeline

3.8 Development Phase Activities

3.9 Agile Iteration Summary

3.10 ToothWise Comprehensive System Test Cases and Results

3.11 ISO/IEC 25010 Software Quality Evaluation Results

3.12 Software Development Requirements

3.13 Implementation Software Requirements

3.14 Hardware Development Requirements

3.15 Implementation Hardware Requirements

3.16 dentists table

3.17 patients table

3.18 screenings table

3.19 detections table

3.20 recommendations table

3.21 clinical_notes table

3.22 audit_logs table

3.23 Implementation / Deployment Procedure

List of Figures

Figure No. Figure Title

1 Agile Development Methodology

2 ToothWise Use Case Diagram
3 Entity Relationship Diagram
4 ToothWise Level 0 Data Flow Diagram
5 ToothWise Level 1 Data Flow Diagram
6 Capstone 1 and Capstone 2 Gantt Chart
7 Public Landing Page
8 Staff Login Page
9 Dashboard
10 Patient List
11 Patient Details
12 New Screening (Capture / Upload)
13 Screening Results
14 Staff Management (Admin)
15 Audit Log (Admin)
16 Patient Portal Login
17 Patient Portal Registration
18 Patient Portal Screening History
19 Three-Tier ToothWise System Architecture
20 ToothWise Screening Process Flow

\List of Appendices

Appendix	Appendix Title	Page No.
A	Evaluation Tools	55
B	Sample Inputs / Output	57
C	Diagrams / illustrations	59

3.4 Hardware Implementation Requirements

CHAPTER I
CAPSTONE PROJECT BACKGROUND

INTRODUCTION

1.1 Project Context

CodiSmile Dental Clinic currently relies on manual visual examination and professional judgment to identify dental caries and other visible dental conditions from patient examinations and intraoral images. Although this approach is essential to dental practice, manually reviewing images may be time-consuming and can result in variations

in assessment and documentation. To support the existing process, the proponents propose ToothWise: Teeth Cavity Detection with Recommendation System Using YOLOv12 for CodiSmile Dental Clinic , an AI-assisted system that analyzes intraoral images and detects possible dental conditions using the YOLOv12 object detection model. The system provides visual detection results, condition labels, descriptions, and preventive recommendations to assist dentists and improve patient understanding. ToothWise is intended to serve as a supplementary screening and documentation tool, while final interpretation and clinical decisions remain the responsibility of licensed dental professionals.

Dental caries, commonly referred to as tooth decay or cavities, remains one of the most prevalent non-communicable health conditions in the world. The World Health Organization (2024) estimates that oral diseases affect nearly 3.7 billion people globally, with untreated dental caries in permanent teeth identified as the single most widespread health condition based on Global Burden of Disease 2021 data. Despite its prevalence, a significant portion of the population, particularly those in geographically isolated or economically disadvantaged communities, continues to face substantial barriers in accessing timely and adequate dental care. The financial cost of consultations, the distance to dental facilities, and low oral health literacy frequently result in delayed care-seeking, by which point cavities have often advanced beyond their most treatable stages. According to Qari et al. (2023), untreated dental caries does not just cause physical pain, it also affects a person's confidence and overall quality of life.

A critical dimension of the current dental care challenge lies in the inconsistency of manual cavity screening practices. The Journal of Evidence-Based Dental Practice (2025) noted that clinical examination outcomes frequently vary depending on the examiner, as different dental practitioners may arrive at differing findings when assessing the same patient for carious lesions. This inter-examiner variability is particularly problematic in high-volume clinic settings where practitioner fatigue and time pressure can further compromise screening reliability. Research has shown that deep learning models

are capable of detecting dental cavities with accuracy that is on par with, or sometimes even better than, trained clinicians, as confirmed by systematic reviews from Schwendicke et al. (2021) and Mohammad Rahimi et al. (2022). This makes AI a genuinely useful tool for supplementing human judgment during screenings.

Intraoral cameras have also changed the way dentists work. Instead of relying purely on what they can see with a mirror and a light, dentists can now capture clear, detailed images of the inside of the mouth. Studies like the one by Kang et al. (2024) show that these images are more than good enough to serve as input for AI-based cavity detection systems, producing consistent and reliable results during actual clinical use. It is this imaging modality that serves as the primary data source for ToothWise during patient screenings at CodiSmile Dental Clinic.

The AI model powering this system is YOLOv12, which was introduced by Tian et al. (2025) as an attention-centric object detection framework. Compared to previous versions in the YOLO family, YOLOv12 uses area attention mechanisms that help it focus selectively on spatially relevant regions, something that happens regularly in dental images where carious lesions appear as small, low-contrast areas across multiple tooth surfaces simultaneously. Its combination of speed and accuracy makes it practical for real-time use during clinical consultations.

This project was created in partnership with CodiSmile Dental Clinic, which serves as the deployment site and clinical reference point throughout the study. The goal is not to take the place of the dentist but to give them a more efficient and objective tool to work with. With ToothWise integrated into the clinic's workflow, cavity screenings can be done faster, findings can be documented more consistently, and patients can walk away with a clearer understanding of their oral health through the system's personalized prevention plan feature.

1.2 Problem Statement

The study addresses the following system-specific problems in the development and evaluation of ToothWise for CodiSmile Dental Clinic:

1. How can ToothWise provide a secure and organized digital workflow for managing patient records, capturing or uploading intraoral images, and creating screening records at CodiSmile Dental Clinic?
2. How can ToothWise integrate the YOLOv8 and YOLOv12 object-detection models to identify and localize trained oral findings in intraoral images, retain each model's results, and present fused clinical findings with bounding boxes and confidence scores?
3. How can ToothWise present detection results, risk information, and dentist notes in a clear screening record that supports licensed dentists in reviewing, documenting, and explaining findings to patients?

4. How can ToothWise generate prioritized, condition- and risk-informed preventive recommendations from the detected findings to support patient oral health education and timely dental consultation?

5. How acceptable is ToothWise to IT experts and clinical end-users at CodiSmile Dental Clinic when evaluated using the ISO/IEC 25010 Software Product Quality Model in terms of functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability?

1.3 Objective of the Study

General Objective

This study aims to develop ToothWise, an AI-assisted dental screening and recommendation system that uses YOLOv12 to analyze intraoral images, records screening results, and provides risk-informed preventive guidance for licensed dentists and patients at CodiSmile Dental Clinic.

The study further aims to evaluate the completed system using the ISO/IEC 25010 Software Product Quality Model to determine its quality and acceptability for its intended clinical workflow.

Specific Objectives

1. 1. To design and implement a secure digital workflow for patient records, intraoral image capture or upload, and screening-record management at CodiSmile Dental Clinic.

2. 2. To integrate the YOLOv8 and YOLOv12 object-detection models to identify and localize trained oral findings in intraoral images, retain their individual results, and present fused clinical findings with bounding boxes and confidence scores.

3. 3. To develop a screening-results interface that displays detected findings, risk information, and dentist notes to support the review, documentation, and patient explanation of screening results.

4. 4. To develop a feature that generates prioritized, condition- and risk-informed preventive recommendations that promote oral health education and timely dental consultation.

5. 5. To evaluate the quality and acceptability of ToothWise among IT experts and clinical end-users using the ISO/IEC 25010 Software Product Quality Model in terms of functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability, and portability.

ISO/IEC 25010 Software Product Quality Framework

The evaluation of ToothWise is anchored on the ISO/IEC 25010 Software Product Quality Model, an internationally recognized software engineering standard governed by the Systems and Software Quality Requirements and Evaluation (SQuaRE) series. ISO/IEC 25010 supersedes the earlier ISO/IEC 9126 standard, establishing a comprehensive taxonomy of eight (8) quality characteristics and sub-characteristics essential for verifying clinical software systems:

1. Functional Suitability. Evaluates the degree to which ToothWise meets specified clinical screening requirements when operated under defined conditions. This encompasses Functional Completeness (full integration of patient records, intraoral image capture/upload, multi-class YOLOv8/v12 cavity and gum inflammation detection, KNN prevention plan recommendation, clinical notes, and PDF reporting), Functional Correctness (precision and recall of detected bounding boxes and 0–100 risk scoring), and Functional Appropriateness (workflow alignment with dental consultation procedures).

2. Performance Efficiency. Assesses the performance of ToothWise relative to the resources consumed. This includes Time Behavior (sub-second AI inference times, with YOLOv12 achieving rapid attention-centric inference and low API response latencies), Resource Utilization (efficient local server CPU/GPU footprint and connection pooling), and Capacity (robust PostgreSQL database performance supporting hundreds of simultaneous screening records and high-resolution images).

3. Compatibility. Measures the capability of ToothWise to function concurrently with other clinic applications. This encompasses Co-existence (running reliably in local server and workstation operating environments alongside clinic management utilities) and Interoperability (seamless JSON-based data exchange across frontend, backend, and intraoral camera USB video interfaces).

4. Usability. Examines the effectiveness, efficiency, and user satisfaction of the system interface. Sub-characteristics include Appropriateness Recognizability (immediate visual clarity of risk badges and bounding boxes), Learnability (intuitive navigation for dental staff and patients), Operability (ergonomic capture and review controls), User Error Protection (Pydantic schema validation, confirmation modals, and boundary enforcement), User Interface Aesthetics (clean, professional CodiSmile design system styling), and Accessibility (high-contrast text and patient-friendly terminology).

5. Reliability. Measures the system's ability to maintain a specified level of operational performance. This includes Maturity (thoroughly debugged codebase verified through automated API testing), Availability (continuous system uptime throughout clinic operating hours), Fault Tolerance (graceful fallback mechanisms, such as deterministic mock inference when model weights are being updated), and Recoverability (automated scheduled database backups using backup_db.py).

6. Security. Ensures that clinical and patient personal data are protected against unauthorized disclosure or tampering. This includes Confidentiality (bcrypt password hashing complying with the 72-byte limit, JWT bearer token authentication separating staff and patient scopes, and authenticated image endpoints preventing public exposure), Integrity (relational constraints and audit logging), Non-repudiation and Accountability (append-only audit trail capturing administrative and clinical actions), Authenticity (strict session and role validation), and Compliance with the Philippine Data Privacy Act of 2012 (Republic Act No. 10173).

7. Maintainability. Reflects the ease with which the software can be modified, enhanced, or adapted. This comprises Modularity (strict three-tier architecture separating React, FastAPI, PyTorch ML models, and PostgreSQL), Reusability (modular UI components and shared schemas), Analyzability (detailed server diagnostics and /api/health endpoint), Modifiability (independent retraining and swapping of YOLOv12 and KNN models without rewriting API logic), and Testability (automated pytest test suites and evaluate_models.py benchmarking).

8. Portability. Evaluates the ease with which ToothWise can be transferred across different deployment environments. This encompasses Adaptability (cross-browser responsiveness across Chrome, Firefox, and Edge), Installability (straightforward virtual environment and database deployment on clinic hardware), and Replaceability (modular configuration files enabling seamless migration to cloud or hybrid infrastructures).

Table 1.1 ISO/IEC 25010 Software Quality Criteria Operationalization Matrix for ToothWise

ISO/IEC 25010 Characteristic	Operational Implementation in Toothwise	Verification Metric / Target Benchmark
1. Functional Suitability	Intraoral image upload/capture, multi-class YOLOv8/YOLOv12 caries & gingivitis detection, 0-100 risk scoring, KNN personalized prevention plan, clinical notes, PDF report.	100% functional pass rate across core screening workflows; zero unhandled clinical exceptions.

2. Performance Efficiency	Area-attention YOLOv12 inference latency, lightweight FastAPI asynchronous endpoints, local PostgreSQL indexing, fast PDF rendering.	Inference latency < 50ms (GPU) / < 1.5s (CPU); API response latency < 200ms for typical screening queries.
3. Compatibility	Standard RESTful JSON API specifications, standard USB/UVC intraoral camera support, multi-platform desktop/tablet browser execution.	Zero co-existence conflicts with Windows 10/11 services; standard web browser execution without plugins.
4. Usability	Color-coded risk level badges (Low=Green, Med=Amber, High=Red), intuitive image annotation canvas, one-click PDF generation, clean CodiSmile UI.	Task completion rate > 95% on first trial by dental staff; Mean Usability rating >= 4.50 on 5-point scale.
5. Reliability	Deterministic mock detector fallback, graceful exception handling, automated PostgreSQL daily backups retaining 14 restore checkpoints.	System availability >= 99.5% during clinic hours; 100% successful database restore from backup scripts.
6. Security	JWT token authentication with separated staff and patient scopes, bcrypt password hashing (72-byte cap), authenticated private image routes, SlowAPI rate limiting (5 req/min).	Zero unauthenticated image access vulnerabilities; 100% rejection of brute-force attempts above rate limit.
7. Maintainability	Three-tier architecture (React, FastAPI, PostgreSQL), decoupled AI weight files (.pt/.joblib), centralized Pydantic schemas, audit logging.	Independent retraining and hot-swapping of AI weights without modifying backend business logic.
8. Portability	Standardized Python 3.11 virtual environment, npm production build, cross-browser compatibility across Chrome, Firefox, and Edge.	Successful local server installation and execution within 30 minutes following standardized deployment guide.

1.4 Scope and Limitation of the Study

The scope of this project covers the development, training, and deployment of an AI-based dental screening system at CodiSmile Dental Clinic. Using live video feeds and still images from a standard intraoral camera, the system will attempt to identify early signs of dental caries and common periodontal anomalies during patient consultations.

The YOLOv12 model will be trained on publicly available annotated dental image datasets that cover healthy teeth, carious lesions, and signs of gum inflammation. Once training is complete, the system will be tested at CodiSmile Dental Clinic to see how it performs in an actual clinical environment. Aside from detection, the system will also generate a risk score and a personalized prevention plan for each patient based on what the model finds.

There are, however, things this system cannot do. It is a support tool , not a replacement for a licensed dentist. The dentist at CodiSmile Dental Clinic will always have the final say on any diagnosis or treatment decision. The prevention plan it generates is meant to guide the patient, not serve as a formal medical prescription. The system also works only with intraoral camera images and does not process X rays or any other radiographic input. It will not detect oral cancer, bone loss, impacted teeth, or orthodontic issues, those remain outside the scope of what visual surface level detection can reliably identify.

1.5 Significance of the Study

This study is expected to be useful to different groups of people in different ways. Patients. For patients of CodiSmile Dental Clinic, the main benefit is getting a faster and more thorough screening experience. Dental problems that might have been missed during a quick manual check can now be flagged by the AI. Getting a clear prevention plan at the

end of the visit also gives patients something concrete to act on, which can make a real difference in their long-term oral health.

Dentists and CodiSmile Dental Clinic. The system helps dentists work smarter. Rather than spending a lot of time on routine visual checks, they can rely on the AI to do the initial screening and then focus their attention on the cases that actually need it. It also improves documentation since every detection result is recorded in the system automatically.

Future Researchers. Students and researchers who want to explore AI in dentistry can use this project as a starting point. The methodology, dataset approach, and system architecture are all things that can be built on or adapted for more advanced future studies.

The Research Community. There are very few local studies that apply AI to dental screening in the Philippine setting. This project adds to that small but growing body of work and can serve as a reference for similar initiatives in other clinics across the country.

1.6 Definition of Terms

Artificial Intelligence (AI). A branch of computer science that focuses on building systems capable of performing tasks that would normally require human thinking, such as identifying objects in images or generating recommendations based on data.

Computer Vision. A field within AI that deals with teaching machines to interpret and make sense of visual data, things like photographs or live video feeds.

Deep Learning. A type of machine learning that uses layered neural networks to process complex patterns. In this project, deep learning is what allows the system to distinguish between healthy teeth, cavities, and inflamed gums in an image.

Dental Caries The clinical term for cavities or tooth decay. It happens when bacteria in the mouth produce acid that gradually eats away at the tooth's enamel and deeper layers.

Flagging. When the system draws a bounding box around a part of the image where it has detected a potential dental anomaly, along with a label identifying what it thinks it found.

Intraoral Camera. A small, handheld camera built to fit inside a patient's mouth. It takes high-resolution images or videos of the teeth and gums that are much clearer than what a dentist can see with the naked eye alone.

Periodontal Anomalies. Problems that affect the gums and the structures that hold teeth in place. Common signs include redness, swelling, bleeding, or early signs of infection.

Risk Categorization. The process of assigning a patient to a risk level , Low, Moderate, or High, based on what the AI detected during the screening and how confident it was in those detections.

YOLOv12 (You Only Look Once Version 12). The detection model used in this project. Introduced by Tian et al. (2025), it is known for being fast enough to work in real time while still being accurate enough for complex multi class detection tasks like identifying dental anomalies in a live camera feed.

ISO/IEC 25010. An internationally recognized standard for Systems and Software Quality Requirements and Evaluation (SQuaRE) that categorizes software product

quality into eight distinct characteristics: Functional Suitability, Performance Efficiency, Compatibility, Usability, Reliability, Security, Maintainability, and Portability. In this study, it provides the foundational evaluation framework for verifying the technical robustness and user acceptability of the ToothWise system.

CHAPTER II

REVIEW OF RELATED LITERATURE AND SYSTEMS

This chapter presents the related literature and studies that have been gathered by the proponents. It introduces many ideas linked to the topic, which includes the research's

principal focus. The study's synthesis is offered to analyze the previous research that the researchers have gathered.

2.1 Related Literature

Convolutional Neural Networks for Dental Image Diagnostics. Schwendicke et al. (2021) put together a systematic review of how convolutional neural networks had been applied in dental imaging, covering everything from caries detection to tooth segmentation. What stood out from their findings was that CNN models were not just competitive with experienced dental clinicians but actually outperformed them in some cases. The authors were careful to point out, though, that the quality of training data matters enormously. Models trained on poorly labeled or inconsistent datasets tend to underperform in real clinical scenarios. They also made it clear that these tools are meant to assist dentists, not replace them. That design principle carries directly into ToothWise, where all detected cavity findings at CodiSmile Dental Clinic are reviewed by a licensed dentist before being communicated to any patient. The review also noted the absence of systems that go beyond detection and actually provide clinical guidance afterward, which is precisely the gap that the personalized prevention plan feature of ToothWise was built to address.

Deep Learning for Caries Detection: A Systematic Review. Mohammad Rahimi et al. (2022) reviewed a large number of studies that used machine learning and deep learning to detect caries across different image types, including intraoral photographs. Their main takeaway was that deep learning, particularly convolutional neural networks, worked reliably for identifying both early and more advanced carious lesions. They also raised a concern that is very relevant to this project: there

are not many large, standardized, and publicly available dental datasets out there, which makes it harder to train models that perform well across different clinical settings. For ToothWise, this limitation was addressed by combining publicly available annotated datasets with data augmentation techniques during model training. The review also found that no existing study had combined automated caries detection with a recommendation output, which reinforces why the prevention plan feature of ToothWise fills a genuine gap in the existing body of work.

Caries Detection with Tooth Surface Segmentation on Intraoral Images Using Deep Learning. Park et al. (2022) took a two-step approach to detecting caries from intraoral photos by first segmenting individual tooth surfaces before running lesion detection on each one separately. By isolating the teeth before looking for cavities, the model produced fewer false positives compared to methods that ran detection on the full image without any preprocessing. More importantly for this study, the research confirmed that photos taken with intraoral cameras are detailed enough to support reliable artificial intelligence-based detection. That finding directly validates the image input approach used in ToothWise, where intraoral camera images serve as the primary data source for the YOLOv12 model during patient screenings at CodiSmile Dental Clinic.

Detecting Dental Caries on Oral Photographs Using Artificial Intelligence. Moharrami et al. (2024) focused specifically on artificial intelligence systems that detect caries from oral and intraoral photographs, which is the exact kind of image that ToothWise works with. Their review covered 19 studies from multiple countries and found that AI models applied to photographic inputs performed consistently well, especially for cavitated lesions that are clearly visible on the tooth surface. Most

significantly, the authors explicitly identified the lack of systems that couple detection with actionable clinical recommendations as a gap that future research needs to address. This is the most direct identification in existing literature of the gap that ToothWise fills. The system does not stop at detection. It generates condition-specific dental health guidance for both the dentist and the patient through the integrated recommendation feature.

Oral Health Philippines Country Profile. The World Health Organization (2022) released a country-specific oral health profile for the Philippines as part of its Global Oral Health Status Report. The numbers are difficult to ignore. Dental caries was listed as one of the most prevalent health concerns in the country, with high rates of untreated cases across all age groups. The report also pointed out something that many Filipinos already know from experience: access to dental care is not evenly distributed. People in rural and community settings have significantly fewer options, and the limited dental workforce makes it harder to reach those who need care the most. This profile is the most authoritative reference for understanding the national oral health landscape that ToothWise seeks to contribute to. Deploying an artificial intelligence-assisted screening system at CodiSmile Dental Clinic is a concrete step toward addressing the access and consistency gaps that the WHO profile describes.

Pediatrician Knowledge, Attitude and Practices on Children's Oral Health in the Philippines. Oliveros Villarico et al. (2023) looked at something that often goes undiscussed, which is the role of pediatricians in preventing dental caries among Filipino children. Their study, conducted at a tertiary public hospital in Metro Manila, found that a significant number of pediatricians had limited awareness of evidence-based caries prevention approaches, even though the 2018 DOH National Survey on Oral Health already documented that 85.2% of Filipino five-year-olds had

dental caries. The gap between what the data shows and what frontline healthcare workers actually know about it is striking. For this project, this literature helps explain why a tool like ToothWise matters at the clinic level. If even pediatricians are not fully equipped to address oral caries early, having an AI-assisted screening system that flags cavities during a regular consultation at CodiSmile Dental Clinic can make a real difference in catching problems before they get worse.

Cost-Effectiveness Analysis of Oral Health Care Package of Services Under PhilHealth. A 2025 health technology assessment published in collaboration with the Department of Health and PhilHealth found that approximately 29 million Filipinos have untreated dental caries, and that investing in early stage dental care at the primary level is actually more cost effective in the long run than dealing with advanced, untreated disease later. The economic argument is straightforward. Catching a cavity early costs far less than treating a severely decayed tooth that now requires extraction or restoration. This literature frames the financial case for why a tool like ToothWise is worth building. Catching cavities early is not just better for patients in terms of comfort and health outcomes. It is also the smarter approach from a public health spending perspective.

Guiding Responsible Artificial Intelligence in Healthcare in the Philippines. Alibudbud et al. (2025) wrote a narrative review and policy analysis on responsible AI adoption in Philippine healthcare. They went through the current landscape of AI applications in the country, examined what regulatory frameworks exist, and laid out recommendations for making sure AI tools in clinical settings are used ethically and safely. Some of the key concerns they raised were limited digital infrastructure, the lack of local clinical validation standards, and the need for healthcare workers to

understand and critically evaluate AI outputs. They also emphasized that AI systems used in Philippine healthcare should be transparent, subject to human oversight, and compliant with the Data Privacy Act of 2012. Reading this study shaped several decisions in how ToothWise was designed, particularly the requirement that all AI detection results at CodiSmile Dental Clinic must be reviewed and confirmed by a licensed dentist before they are communicated to patients, and that all patient data is stored securely in a locally managed database.

2.2 Related Studies

Detecting White Spot Lesions on Dental Photography Using Deep Learning. Askar et al. (2021) worked on detecting white spot lesions in intraoral dental photos using deep learning. White spot lesions are early stage demineralization, essentially the very beginning of a cavity before it becomes a visible hole. What made this study particularly useful is that it was done with a small dataset, which is a realistic constraint for many research projects, including this one. Despite that limitation, the model still managed to detect subtle surface level changes that would be easy to miss in a routine visual check. For ToothWise, this matters because it shows that even with limited annotated data, deep learning can pick up early, crucial changes from photographs, which is the same image modality the system uses at CodiSmile Dental Clinic. It also gave the team confidence that training a YOLOv12 model for early cavity detection is achievable even when the training dataset is not massive.

Ultralytics YOLO (Version 8.0.0). Jocher et al. (2023) released YOLOv8 as an open source framework under the Ultralytics platform. Compared to older YOLO versions, YOLOv8 came with a cleaner training pipeline, an anchor free detection head, and support for multiple tasks like detection, segmentation, and classification all in one package. It became one of the most widely used detection frameworks in both academic and applied settings because it was relatively easy to work with while

still delivering strong results. Understanding YOLOv8 matters for this project because YOLOv12 was built on the same foundation. The training tools, dataset formats, and evaluation methods used in YOLOv8 largely carry over to YOLOv12, so much of what Jocher et al. (2023) established applies directly to how the ToothWise model was trained and tested.

Diagnostic Accuracy of Dental Caries Detection Using Ensemble Deep Learning with Intraoral Camera Images. Kang et al. (2024) took a different approach from most dental AI studies by using an ensemble of deep learning models rather than a single model for caries detection. By combining multiple detectors, they were able to reduce false positives while keeping sensitivity high, which is especially important in a clinical setting where unnecessary alarms can make dentists lose trust in the system. Their work was done using actual intraoral camera images from a real dental practice, not just radiographs, which makes it closely comparable to what ToothWise is doing. The models were validated against annotations made by experienced dentists, and the results held up well. This study gave the team confidence that deep learning for real-time intraoral caries detection is not just theoretically possible but has already been shown to work in a clinical environment similar to CodiSmile Dental Clinic.

AI-Assisted Detection of Interproximal, Occlusal, and Secondary Caries on Bite-Wing Radiographs Using YOLOv8. Karakus et al. (2024) applied YOLOv8 to the task of detecting different types of dental caries from bitewing radiographs. One thing that stood out in their findings was how well YOLOv8 handled small and overlapping lesions, which is a tricky problem in dental imaging because caries can be close together or partially hidden. Even though ToothWise uses intraoral

photographs rather than radiographs, the core detection challenge is similar. The model needs to find and correctly label multiple small, curious areas within a single image. The performance of YOLOv8 in this study, combined with the architectural improvements introduced in YOLOv12, provided the team with a strong technical basis for believing that YOLOv12 can handle the cavity detection demands of ToothWise.

YOLOv12: Attention-Centric Real-Time Object Detectors. Tian et al. (2025) introduced YOLOv12 in a preprint that gained significant attention in the computer vision community. The paper presents a new approach to the YOLO architecture centered around area attention mechanisms, which allow the model to focus on relevant regions of an image rather than processing everything equally. The results were impressive. The YOLOv12 N configuration achieved 40.6% mean average precision at an inference latency of just 1.64 milliseconds on a T4 GPU, outperforming comparable YOLOv10 and YOLOv11 models. For a system like ToothWise that needs to run in real time during a dental consultation, that combination of accuracy and speed is exactly what is needed. This paper is the primary technical reference for the detection model used in this project, and its findings directly justify choosing YOLOv12 over earlier versions for deployment at CodiSmile Dental Clinic. It is also worth noting that no existing study has applied YOLOv12 to dental cavity detection, making ToothWise the first to explore this architecture in an intraoral dental screening context.

Bridging the Gap: A Call for Capacity Building in AI for Healthcare in the Philippines. A study from the University of Santo Tomas (2023) took a critical look at AI readiness in Philippine healthcare. When the researchers tested AI generated responses to clinical queries relevant to Filipino patients, they found notable inaccuracies, including instances where the AI produced fabricated references and

incorrect medical guidance. The paper used this to argue that Filipino healthcare professionals urgently need better AI literacy and that unsupervised use of AI clinical tools carries real risks. The call to action was for structured training and, most importantly, mandatory human oversight in any clinical AI application. This study speaks directly to a design decision in ToothWise. The system does not allow AI detection results to be shown to patients without going through a dentist first. At CodiSmile Dental Clinic, the dentist reviews every output before it becomes part of the clinical record or is communicated to the patient, which is exactly the kind of human oversight that the UST study argued for.

Synthesis

The literature and studies reviewed in this chapter come together to build a clear and well-supported case for the approach that ToothWise has taken. Schwendicke et al.

(2021) and Mohammad Rahimi et al. (2022) established through separate systematic reviews that deep learning models are genuinely reliable for dental caries detection, often performing at the level of trained clinicians or better. Park et al. (2022) and Kang et al. (2024) both confirmed that intraoral photographs are a sufficient and practical input modality for AI-based cavity detection, which directly validates the image acquisition approach of ToothWise at CodiSmile Dental Clinic.

The most important gap identified across the reviewed works was stated most directly by Moharrami et al. (2024): existing dental AI systems detect cavities but stop there. None of the systems reviewed generated recommendations or patient facing guidance after detection. ToothWise fills this gap by embedding a structured recommendation engine into the detection pipeline, making it the first system to pair YOLOv12-based cavity detection with an actionable personalized prevention plan.

On the technical side, Tian et al. (2025) established YOLOv12 as the most accurate and fastest real-time object detection model currently available, while Jocher et al. (2023) and Karakus et al. (2024) documented the strong performance of YOLO family architectures in dental detection tasks. Askar et al. (2021) further showed that deep learning cavity detection is achievable even under limited dataset conditions, supporting the training strategy used in this project.

The Philippine context literature from the WHO (2022), Oliveros Villarico et al. (2023), and the PhilHealth cost-effectiveness analysis (2025) grounds all of this in a concrete public health reality. Dental caries is widespread among Filipinos, it often go undetected until it has already progressed, and access to professional dental care remains out of reach for many due to cost and geography. Alibudbud et al. (2025) and the UST (2023) study shaped responsible design decisions around human oversight, data privacy, and clinical transparency. What no existing work has done is bring together YOLOv12-based real-time cavity detection, a personalized prevention plan generator, and a full-stack

clinical system deployed at an actual Philippine dental clinic. That is the specific contribution of ToothWise.

…

CHAPTER III
METHODOLOGY

This chapter presents the methodology used to develop ToothWise for CodiSmile Dental Clinic. It describes the Agile development process, project planning, team responsibilities, requirements analysis, system design, development, and testing. It also documents the system's software and hardware requirements, architecture, database, AI and machine-learning components, risk and recommendation engines, security controls, and deployment specifications.

3.1 Development Methodology

ToothWise was developed using the Agile Software Development Life Cycle (Agile SDLC). Agile was selected because the system integrates several interconnected components, including a React.js web interface, a FastAPI REST backend, a PostgreSQL database, a dual-model YOLOv8/YOLOv12 detection pipeline, a KNN-based recommendation engine, authentication and authorization, audit logging, and a separate patient self-service portal. These components benefit from incremental development, continuous testing, regular review, and repeated refinement rather than a single sequential build.

Instead of completing the entire system in one long development cycle, the project was carried out through manageable iterations in which requirements were prioritized, features were implemented, tested, reviewed, and improved before the next set of features was developed. Each iteration followed the same general cycle of planning, requirements analysis, system design, development, testing, evaluation, and refinement, illustrated in Figure 1.

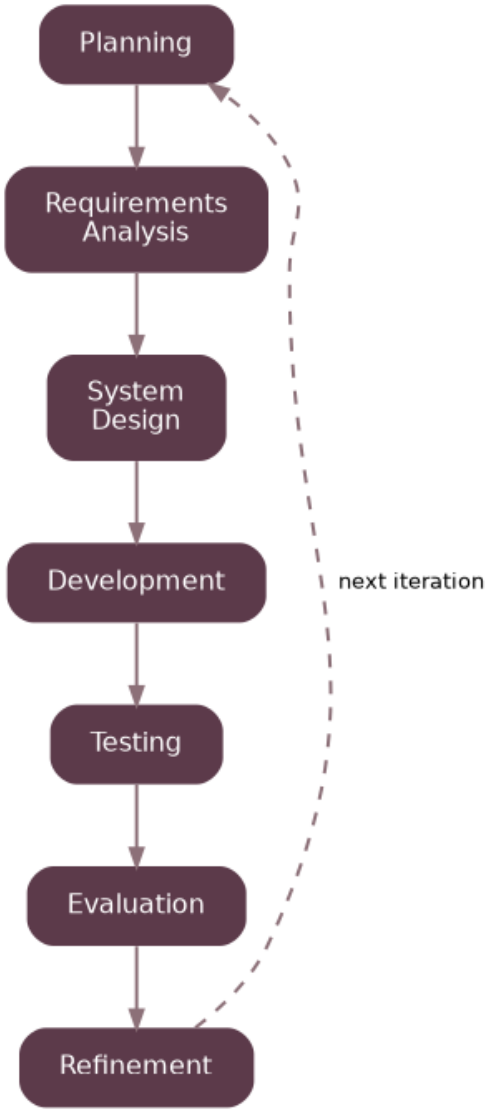


Figure 1 Agile Development Methodology

The cycle in Figure 1 was applied repeatedly across the project's documented iterations. Planning and requirements analysis identified the features required for the clinic workflow; system design translated those requirements into frontend routes, API endpoints, and database structures; development implemented the corresponding React, FastAPI, and machine-learning components; testing verified the implemented behavior;

evaluation compared the result against the intended requirements; and refinement corrected any issues found before the next iteration began.

3.2 Project Planning

The ToothWise project was planned around the screening and record-keeping workflow of CodiSmile Dental Clinic. The following considerations guided the planning of the system:

- Project Objectives — to provide CodiSmile Dental Clinic with an AI-assisted dental screening system that detects possible cavities and related findings from an intraoral image, estimates a risk level, and generates a recommendation/prevention plan, while also supporting day-to-day clinic record-keeping.
- Project Scope — the system covers staff authentication, patient management, dental screening and AI detection, risk assessment, recommendation generation, clinical notes, PDF reporting, audit logging, and a self-service patient portal.
- Target Users — dentists and administrators at CodiSmile Dental Clinic, and the clinic's patients through the patient portal.
- Clinic Workflow — a patient is registered or already on file, an intraoral image is captured or uploaded during a screening, the system produces detection, risk, and recommendation results for the dentist to review, and the dentist may add clinical notes or generate a PDF report.
- Functional Requirements — detailed in Section 3.4.
- Non-Functional Requirements — detailed in Section 3.4, covering security, usability, reliability, performance, and data protection.
- AI Screening Requirements — the system was planned to use two independent detection models (YOLOv8 and YOLOv12) whose outputs are fused into a single clinical detection set, rather than relying on a single model.

- Database Requirements — a relational PostgreSQL schema was planned to store dentists, patients, screenings, detections, recommendations, clinical notes, and audit logs with referential integrity between related records.
- Security Requirements — JWT-based authentication, hashed passwords, rate-limited login endpoints, authenticated (non-public) image access, and audit logging were planned from the outset given the clinical and personal nature of the data involved.

The requirements and technical decisions behind this planning were drawn directly from the current ToothWise project package rather than from separate interview transcripts or survey forms, since the project does not maintain those as separate dated records. Table 3.1 summarizes the actual sources used.

Table 3.1 Sources of Requirements and Technical Data

Source	Use in the Study
System and code review	The frontend routes, pages, API client, FastAPI routers, schemas, security module, ML modules, and database models were reviewed to identify actual implemented functions and data flows.
Database and schema review	The PostgreSQL schema and migration scripts were reviewed to verify entities, relationships, enumerated values, detection sets, and audit logs.
Testing and debugging records	CHANGES.md documents API contract reconciliation, enum and image-path fixes, smoke testing, security validation, and feature verification.
AI evaluation procedure	The evaluate_models.py utility defines how precision, recall, F1, mAP50, mAP50-95, and average inference time can be obtained from a labeled test set. No real model comparison

	values are claimed here since the current package documents the use of mock inference until trained weights are installed.

3.3 Roles and Responsibilities of the Four-Member Development Team

ToothWise was developed by a four-member capstone development team. Table 3.2 presents each member's assigned role and corresponding responsibilities based on the work represented in the current project package. A member may carry more than one responsibility.

Table 3.2 Roles and Responsibilities of the Development Team

Group Member	Assigned Role	Responsibilities
Arreola, Reizen S.	Project Manager / System Analyst	Coordinated the activities of the development team, monitored the project schedule, translated clinic workflow requirements into system requirements, and ensured that project deliverables remained aligned with the objectives of the Toothwise system.
Magpantay, Rizelle Carla N.	Programmer / Backend Developer	Implemented and integrated the FastAPI REST API, authentication and authorization logic, and database operations and connected the frontend with the backend API; resolved programming and integration issues identified during testing.
Sales, Keith Ashly V.	Frontend Developer / UI-UX Designer	Implemented the React.js user interface, including the dashboard, patient and

		dentist management screens, screening workflow, patient portal, and the visual design and layout of the system.
Tubillas, Brayan Axel L.	AI/ML Developer / Database Developer / QA	Developed and integrated the YOLOv8/YOLOv12 dental image detection pipeline, the risk engine, and the KNN recommendation engine; designed and maintained the PostgreSQL database schema and relationships; and conducted functional, security, and integration testing of the system.

The specific division of responsibilities above reflects the major components of the current system — project coordination and requirements, backend/API development, frontend/interface development, and AI/database/quality assurance work. Where an exact individual assignment could not be independently confirmed from the available project files, the responsibility is described at the level of the actual work represented in the system rather than assumed.

3.4 Requirements Analysis

The requirements of ToothWise were analyzed from the intended clinic workflow and the functions represented in the current system and its API. ToothWise has two authenticated user groups: clinic-side users (dentists and administrators) and patients using the self-service portal. Tables 3.3 and 3.4 present the functional and non-functional requirements that are currently implemented in the application and verified against the project source code.

Table 3.3 Functional Requirements

ID	Requirement	Specification
FR-01	Staff authentication	Authenticate dentists and administrators by email and password and issue a JWT bearer token for authenticated API access.
FR-02	Role-based access	Distinguish administrator, dentist, and patient access scopes and restrict protected functions accordingly.
FR-03	Patient management	Create, view, update, and delete patient records and view screening history and risk trends.
FR-04	Dental screening	Submit an intraoral image in a single multipart screening request and receive detection, risk, recommendation, and screening-record results.
FR-05	AI detection	Run the configured YOLOv8 and YOLOv12 detectors, retain raw model outputs, and produce a fused clinical detection set.
FR-06	Risk assessment	Calculate a 0-100 risk score and classify the screening as LOW, MEDIUM, or HIGH.
FR-07	Recommendation generation	Use the KNN recommender and condition-specific rules to generate an ordered prevention plan.
FR-08	Screening review	View screening images, fused detections, risk information, recommendations, clinical notes, and model-comparison information.
FR-09	PDF report	Generate an authenticated PDF screening report containing patient/screening information, findings, prevention plan, and clinical notes.
FR-10	Patient portal	Patients register, log in, and view their own screening history and screening details.
FR-11	Audit logging	Record relevant authentication and create/update/delete actions for administrative review.

FR-12	Health monitoring	Expose a health endpoint that checks database connectivity without exposing internal errors.
FR-13	Database backup	Provide a pg_dump-based backup utility that creates timestamped, retained backups.

Table 3.4 Non-Functional Requirements

ID	Quality Area	Specification
NFR-01	Security	Passwords hashed with bcrypt; protected operations require JWT authentication; login/registration endpoints rate-limited to 5 requests per minute per IP.
NFR-02	Privacy	Screening images are not publicly exposed; authenticated and ownership-checked image endpoints are used instead of a public upload route.
NFR-03	Usability	Browser-based workflows for dashboard monitoring, patient records, screening, dentist management, audit review, and patient self-service.
NFR-04	Reliability	Backend returns a generic response for unhandled errors while recording diagnostic detail server-side.
NFR-05	Performance	YOLOv8 and YOLOv12 inference times are recorded per screening; model-comparison views report runtime and detection statistics.
NFR-06	Data Protection	Relational constraints, enumerated status/risk values, foreign keys, and transactions preserve consistent, valid records.

3.5 System Design

This section presents the major design diagrams required for the capstone preliminary examination: the Use Case Diagram, the Entity Relationship Diagram, the Data Flow Diagrams at Level 0 and Level 1, and the development Gantt Chart. Each diagram was produced from the actual implemented system roles, database schema, and documented development sequence.

3.5.1 Use Case Diagram

Figure 2 presents the use case diagram of ToothWise. It represents the system's user roles Administrator, Dentist, and Patient—and the functions each role can currently perform, verified against the implemented frontend routes and backend authorization checks. This diagram represents system user roles, not the four-member development team presented in Section 3.3.

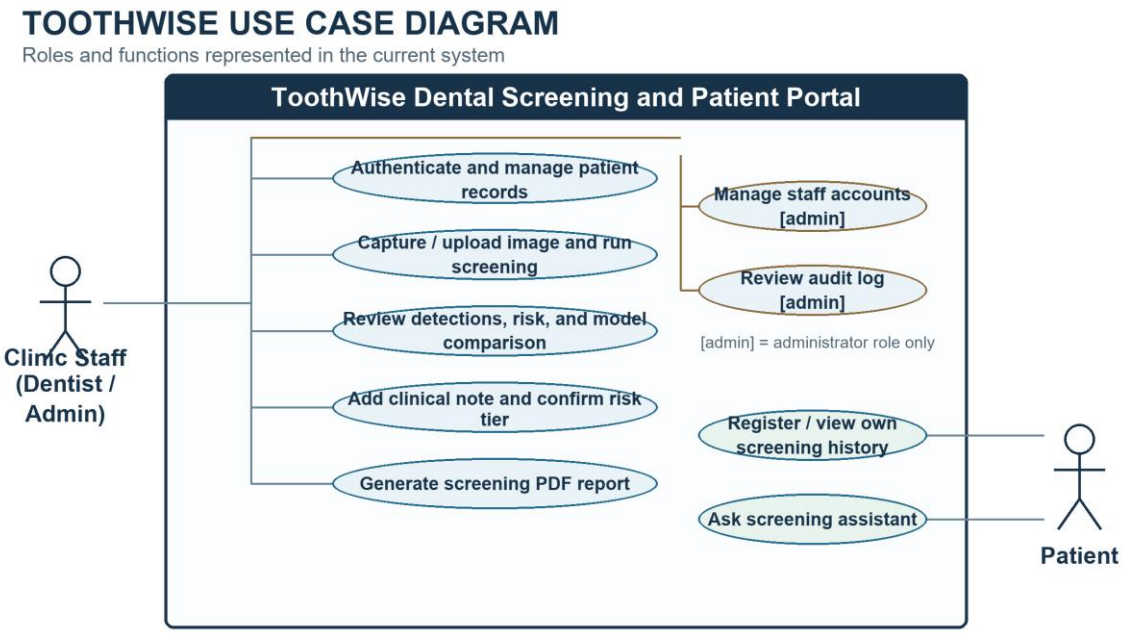


Figure 2 ToothWise Use Case Diagram

3.5.2 Entity Relationship Diagram (ERD)

Figure 3 presents the Entity Relationship Diagram of the ToothWise PostgreSQL database, verified against the current database/schema.sql and backend/app/models.py. The database contains seven tables: dentists, patients, screenings, detections, recommendations, clinical_notes, and audit_logs, related through foreign keys as shown below. The complete field-level specification of each table is presented in Section 3.9.4.

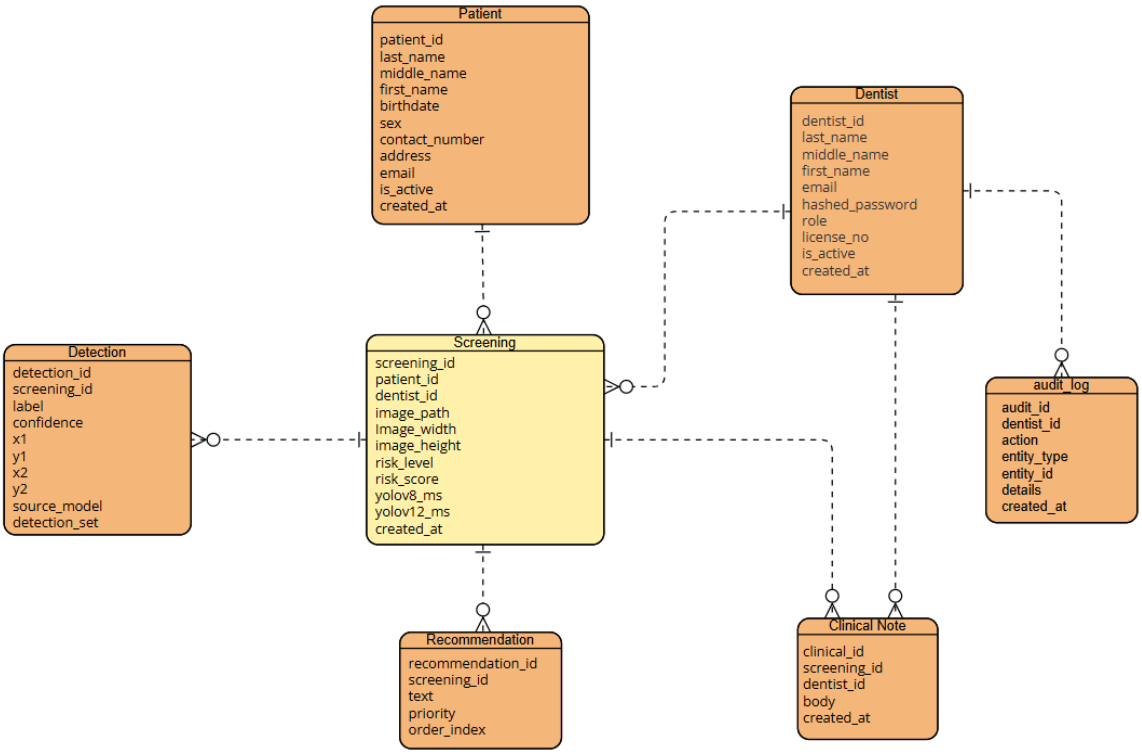


Figure 3 Entity Relationship Diagram (ERD)

Table 3.5 consolidates the seven core entities of the ToothWise database into a single fact table, summarizing each table's primary key, purpose, and relationships to the other entities, ahead of the full field-level data dictionary presented in Section 3.9.4.

Table 5 ToothWise Database Fact Table

Table	Primary Key	Purpose	Related Tables
dentists	id	Stores staff accounts (dentist or admin role) used for staff login and role-based access control.	screenings, clinical_notes, audit_logs
patients	id	Stores patient demographic information and, optionally, patient-portal login credentials.	screenings
screenings	id	Central record of one AI-assisted dental image screening event, including risk level and model timing.	patients, dentists, detections, recommendations, clinical_notes
detections	id	Stores each bounding-box finding produced by the YOLOv8/YOLOv12 ensemble for a screening (5-label taxonomy).	screenings
recommendations	id	Stores the prevention/advice text generated by the KNN recommender for a screening.	screenings
clinical_notes	id	Stores dentist-authored clinical notes attached to a screening.	screenings, dentists

audit_logs	id	Append-only record of sensitive staff actions, kept for administrative review.	dentists

Data Flow Diagram Level 0

Figure 4 presents the Level 0 (context) Data Flow Diagram of ToothWise, designed in strict compliance with Gane and Sarson notation rules. In accordance with Gane and Sarson conventions, the system boundary is represented as a single central process (Process 0.0) rendered as a rounded rectangle with an upper identification header band and a lower descriptive body. The three interacting external entities—System Administrator, Licensed Dentist / Clinical Staff, and Patient—are depicted as distinct double-bordered / shaded rectangular sources and sinks. All communications are modeled through directed, labeled data flow arrows bearing explicit noun phrases, ensuring that no direct entity-to-entity data flows occur without passing through the central system process.

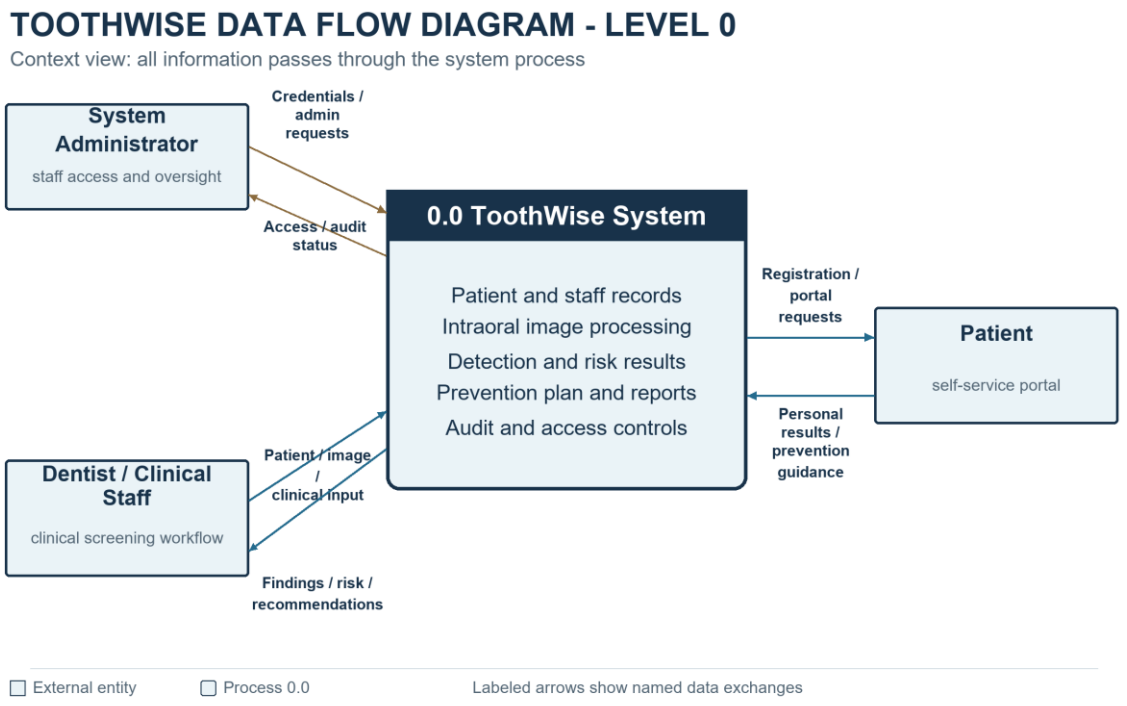


Figure 4 ToothWise Level 0 Data Flow Diagram (Context Diagram) — Gane and Sarson Notation

Data Flow Name	Source	Destination	Description / Transferred Data Elements
Staff credentials and sign-in request	Dentist / Administrator	ToothWise System (0.0)	Staff email and password submitted for authentication and role-scoped access.
Staff management and audit queries	System Administrator	ToothWise System (0.0)	Staff account create, update, or deactivation requests and audit-log queries.
Patient profile and screening request	Dentist / Clinical Staff	ToothWise System (0.0)	Patient demographic details and request to start or review a screening.
Intraoral image	Dentist / Intraoral Camera	ToothWise System (0.0)	Captured camera frame or uploaded

			intraoral image for analysis.
Detection and risk results	ToothWise System (0.0)	Dentist / Clinical Staff	Detected labels, bounding boxes, confidence values, model timings, and screening risk level.
Clinical note and tier confirmation	Dentist / Clinical Staff	ToothWise System (0.0)	Dentist-authored screening note or confirmation of the AI risk tier.
Screening report	ToothWise System (0.0)	Dentist / Clinical Staff	Authenticated PDF report containing the screening image, findings, risk level, recommendations, and notes.
Audit trail and service status	ToothWise System (0.0)	System Administrator	Recorded sign-in and sensitive record actions, plus service and database health status.
Patient registration and portal request	Patient	ToothWise System (0.0)	Self-registration or login, own-history request, and question submitted to the screening assistant.
Patient screening history and guidance	ToothWise System (0.0)	Patient	The patient's own screening findings and recommendations, history, and assistant response.

Table 3.6 Data Flow Dictionary for ToothWise Level 0 Context Diagram

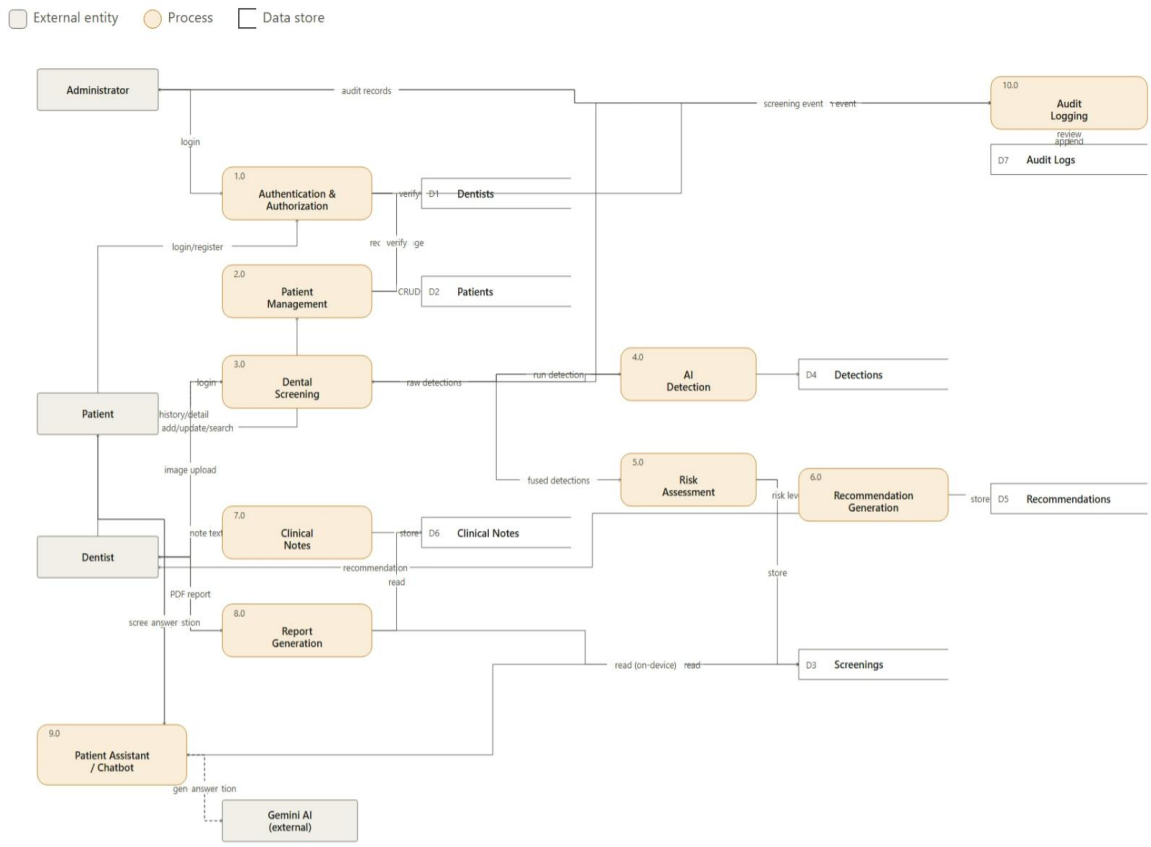
As illustrated in Figure 4, the Level 0 Context Data Flow Diagram establishes the boundary of the ToothWise system (Process 0.0) and details all primary data exchanges with its three external entities: Licensed Dentist / Clinical Staff, System Administrator, and Patient. The system serves as an intelligent intermediary, ingesting raw intraoral clinical inputs, executing dual-model AI inference and risk scoring, and producing actionable diagnostic feedback, authenticated screening summaries, and administrative

audit records. Table 3.6 presents the comprehensive Data Flow Dictionary for the Level 0 diagram.

Data Flow Diagram Level 1

Figure 5 decomposes the ToothWise system into its ten major processes: Authentication and Authorization, Patient Management, Dental Screening, AI Detection, Risk Assessment, Recommendation Generation, Clinical Notes, Report Generation, Patient Assistant/Chatbot, and Audit Logging, together with their associated data stores. This diagram is consistent with the Level 0 diagram in Figure 4 and with the ERD in Figure 3.

Figure 5 Data Flow Diagram Level 1



Gantt Chart

The development schedule of ToothWise spanned thirty-three (33) academic weeks across Academic Year 2025–2026, systematically partitioned into Capstone Project 1 and Capstone Project 2 in accordance with the Bachelor of Science in Information System curriculum requirements:

• Capstone Project 1 (1st Semester: September 1, 2025 – December 19, 2025 | Weeks 1–17):

Encompassed project initiation, clinical requirements analysis with CodiSmile Dental Clinic, architectural modeling (Gane and Sarson DFDs, ERD, and Use Case specifications), initial UI/UX wireframing, PostgreSQL schema initialization, core FastAPI backend setup (Iteration 1), and preliminary YOLOv8/v12 model training and feasibility evaluation, culminating in Milestone 1 (Proposal Defense, October 2025) and Milestone 2 (Midterm Architecture & Prototype Walkthrough, December 2025).

• Capstone Project 2 (2nd Semester: January 5, 2026 – April 15, 2026 | Weeks 18–33): Encompassed the full implementation of the dual-model AI detection pipeline and ensemble fusion (Iteration 2), the auditable 0–100 risk calculation engine and KNN recommendation system (Iteration 3), complete clinical workflow integration with live intraoral camera capture, ReportLab PDF generation, and the patient self-service portal (Iteration 4), extensive security hardening and automated TestClient verification (Iteration 5), clinical pilot deployment at CodiSmile Dental Clinic, and rigorous multi-stakeholder software quality evaluation based on the ISO/IEC 25010 Product Quality Model, concluding with Milestone 3 (Pre-Oral Defense, March 2026) and Milestone 4 (Final Oral Defense and System Turnover, April 15, 2026).

Figure 6 presents the comprehensive Capstone 1 and Capstone 2 Gantt Chart and milestone progression, and Table 3.7 details the Work Breakdown Structure (WBS) with corresponding deliverables across both semesters.

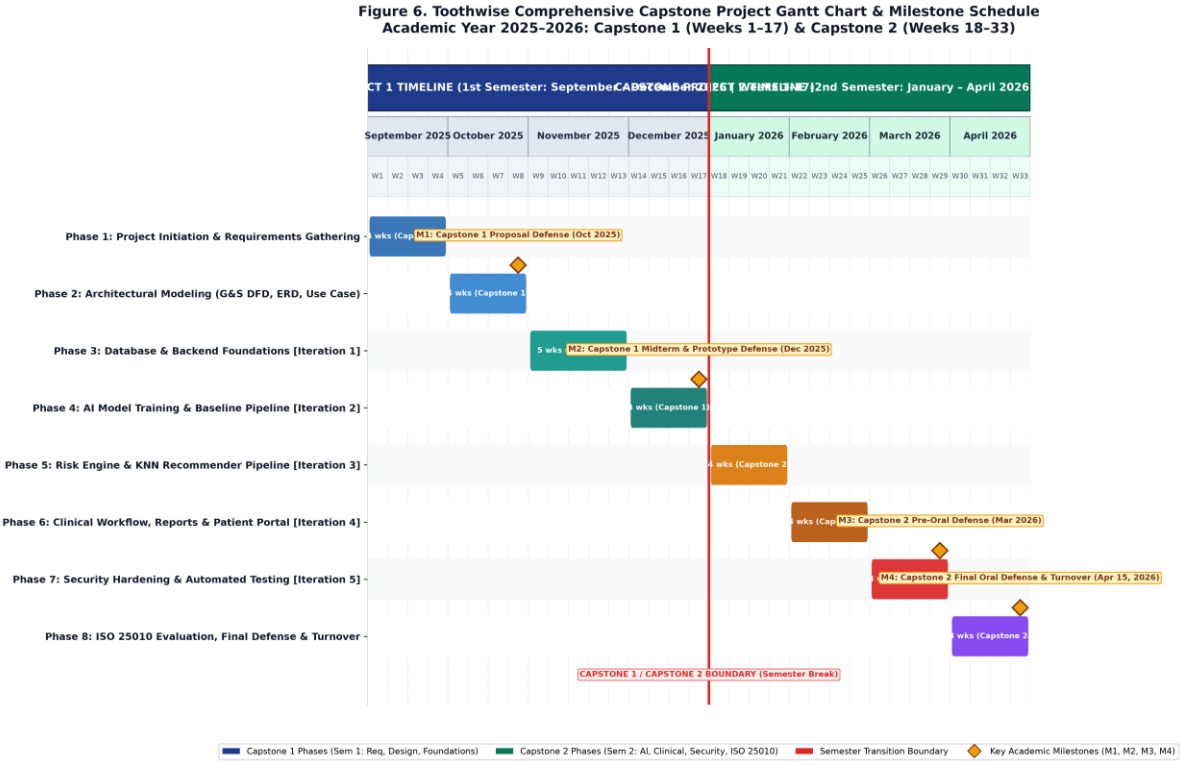


Figure 6 ToothWise Capstone 1 & Capstone 2 Development Gantt Chart and Milestone Schedule (AY 2025–2026)

Table 3.7 ToothWise Capstone Project Work Breakdown Structure (WBS) and Timeline

Capstone Term	Phase & Focus	Timeline / Weeks	Agile Iteration	Key Tasks & Activities	Lead	Milestone Deliverable
Capstone 1 (1st Sem)	**Phase 1: Project Initiation & Requirements Gathering**	Weeks 1–4 (Sept 1 – Sept 28, 2025)	Iteration 1 Foundation	Clinic consultations with CodiSmile, workflow mapping, oral health context analysis, hardware/software feasibility study.	Arreola (PM/SA)	Approved Project Charter & Requirements Matrix
Capstone 1 (1st Sem)	**Phase 2: Architectural Modeling & System Design**	Weeks 5–8 (Sept 29 – Oct 26, 2025)	Iteration 1 Foundation	UI/UX wireframing, Use Case modeling, ERD database design, Gane & Sarson DFD Level 0 & Level 1 design, REST API specs.	Sales (UI/UX) & Tubillas (DB)	System Design Document & Proposal Defense (M1)
Capstone 1 (1st Sem)	**Phase 3: Database & Backend Foundations**	Weeks 9–13 (Oct 27 – Nov 30, 2025)	Iteration 1 Core System	PostgreSQL schema implementation, FastAPI framework	Magpantay (Backend) & Tubillas (DB)	Working API Foundation & Authentication Module

				initialization, JWT auth & RBAC, patient/dentist CRUD endpoints.		
Capstone 1 (1st Sem)	**Phase 4: AI Model Training & Baseline Pipeline**	Weeks 14–17 (Dec 1 – Dec 19, 2025)	Iteration 2 AI Screening	Dental dataset acquisition & augmentation, YOLOv8 & YOLOv12 baseline training, preliminary model weight verification.	Tubillas (AI/ML) & Magpantay (Backend)	Trained Baseline Weights & Midterm Defense (M2)
Capstone 2 (2nd Sem)	**Phase 5: Risk Engine & KNN Recommendation System**	Weeks 18–21 (Jan 5 – Feb 1, 2026)	Iteration 3 Clinical Workflow	Confidence-weighted risk scoring algorithm, KNN recommender training (K=5), prevention plan generator, clinical notes authoring.	Tubillas (AI) & Sales (Frontend)	Clinical Recommendation & Risk Scoring Engine
Capstone 2 (2nd Sem)	**Phase 6: Clinical Workflow, Reports & Patient Portal**	Weeks 22–25 (Feb 2 – Mar 1, 2026)	Iteration 4 Clinic Operations	React camera live capture/upload, screening results review canvas, ReportLab PDF generation, self-service patient portal & chatbot.	Sales (Frontend) & Magpantay (Backend)	Complete End-to-End Clinic & Portal Workflow
Capstone 2 (2nd Sem)	**Phase 7: Security Hardening & Automated Testing**	Weeks 26–29 (Mar 2 – Mar 29, 2026)	Iteration 5 Hardening	SlowAPI rate limiting (5 req/min), bcrypt 72-byte cap, authenticated image endpoints, TestClient automated test suites, bug fixes.	Tubillas (QA) & Magpantay (Sec)	Hardened System & Pre-Oral Defense (M3)
Capstone 2 (2nd Sem)	**Phase 8: ISO/IEC 25010 Evaluation & Final Turnover**	Weeks 30–33 (Mar 30 – Apr 15, 2026)	Iteration 5 Closure	Evaluation survey administration (IT experts & dentists), statistical data analysis, final paper documentation, clinic deployment.	Full Capstone Team	Final Oral Defense (M4) & Clinic Handover (Apr 15)

System Development

ToothWise was developed iteratively, following the six development phases in Table 6, applied repeatedly across the five documented iterations in Table 7 rather than as a single sequential build.

Table 3.8 Development Phase Activities

Phase	Activities
1. Planning and Requirements	Define the clinic workflow, user roles, screening objectives, AI detection needs, recommendation output, patient portal functions, security requirements, and reporting needs.
2. System Analysis	Translate the requirements into frontend routes, FastAPI endpoints, database entities, AI modules, authentication dependencies, and data flows.
3. System Design	Design the three-layer architecture, relational schema, JWT authentication model, detection-set structure, recommendation flow, patient ownership checks, and local network deployment.
4. Development	Implement React pages and API services, FastAPI routers and schemas, SQLAlchemy models, PostgreSQL schema, YOLO detector integration, KNN recommender, risk engine, reports, audit logs, and patient portal.
5. Integration and Testing	Reconcile frontend requests with the actual backend contract, validate database enum labels and image paths, smoke-test API workflows, test security controls, and verify the frontend build.
6. Deployment and Maintenance	Prepare the PostgreSQL database, backend environment, frontend build, model-weight configuration, local network access, backups, and operational monitoring. Continue maintenance through versioned changes and database migration scripts.

Table 3.9 Agile Iteration Summary

Iteration / Focus	Major Activities	Output
Iteration 1 – Core System Foundation	Initial React.js frontend, FastAPI backend, PostgreSQL schema, authentication, and core patient/dentist structures.	Working application foundation and authenticated API.
Iteration 2 – AI Screening	Integration of intraoral image upload, YOLO detection workflow, detection storage, risk scoring, and screening results.	End-to-end dental screening workflow.
Iteration 3 – Recommendations and Clinical Workflow	KNN recommendation logic, prevention plans, clinical notes, screening details, and PDF reporting.	Expanded clinical screening and recommendation workflow.
Iteration 4 – Clinic Operations	Dashboard functions, dentist management, audit logs, and patient portal features.	Broader clinic and patient workflows.
Iteration 5 – Testing and Hardening	API contract reconciliation, database and enum corrections, image access controls, rate limiting, validation, security headers, and frontend integration fixes.	Hardened and verified system behavior.

Each iteration in Table 7 was carried through the full cycle shown earlier in Figure 1, the corresponding requirements were planned and analyzed, the affected parts of the system

were designed, the frontend and backend components were developed and integrated, the iteration's features were tested, the result was evaluated against the intended requirement, and any issue found was corrected before the next iteration began. The Gantt chart in Figure 6 visualizes this same sequence.

The system was implemented as a three-tier web application: a React.js single-page frontend, a FastAPI REST backend exposing versioned JSON endpoints, and a PostgreSQL relational database. The AI detection pipeline (YOLOv8 and YOLOv12) and the KNN recommendation engine were developed as backend modules invoked synchronously by the screening endpoint. The complete technical specification of this architecture is presented in Chapter III.

3.7 System Testing

ToothWise was tested through functional, integration, and security testing conducted against the running backend using FastAPI's TestClient, supplemented by manual and code-review verification for checks that are not easily exercised through an automated client alone. Table 3.10 presents the test cases executed against the current system and their documented results.

Table 3.10 ToothWise Comprehensive System Test Cases and Verification Results

Test ID	Module / Area	Test Scenario	Pre-conditions & Test Inputs	Expected Result	Actual Documented Result	Status
TC-01	Authentication	Staff Login - Valid Credentials	Registered staff account; POST /api/auth/login with valid email and password.	HTTP 200 OK; returns valid JWT access token and user role profile.	Verified successful login and JWT bearer token issuance.	**PASSED**
TC-02	Authentication	Staff Login - Invalid Password	Registered staff account; POST /api/auth/login with invalid password.	HTTP 401 Unauthorized; generic error message without disclosing user existence.	Verified HTTP 401 generic invalid-credential response and audit logging.	**PASSED**
TC-03	Access Control	Role-Based Access Control	Authenticated dentist session; attempt GET /api/dentists/ or /api/audit/.	HTTP 403 Forbidden; admin-only endpoints strictly	Verified HTTP 403 response; dentist restricted to clinical workflows.	**PASSED**

		(Dentist vs Admin)		blocked for dentist role.		
TC-04	Staff Management	Admin Creates New Dentist Account	Authenticated admin session; POST /api/dentists/ with valid name, email, role, license.	HTTP 201 Created; dentist record persisted with bcrypt hashed password.	Verified dentist profile creation and persistence in PostgreSQL.	**PASSED**
TC-05	Patient Management	Register New Patient Record	Authenticated staff session; POST /api/patients/ with valid demographic vitals.	HTTP 201 Created; patient record stored with unique patient_id.	Verified patient profile correctly created and searchable.	**PASSED**
TC-06	Patient Management	Update Patient Contact and Address	Existing patient record; PUT /api/patients/{id} with modified phone and address.	HTTP 200 OK; patient details updated with timestamp refresh.	Verified patient profile updated and audit log event generated.	**PASSED**
TC-07	Patient Management	Patient Soft Deletion / Deactivation	Existing patient with screening records; DELETE /api/patients/{id}.	HTTP 200 OK; is_active set to false; clinical screening records preserved.	Verified referential integrity preserved; historical screenings intact.	**PASSED**
TC-08	Image Acquisition	Upload Intraoral JPEG Image	Valid patient_id; multipart POST /api/screenings/ with valid JPEG photo.	HTTP 201 Created; image saved to disk with unique hashed filename; screening created.	Verified image stored securely; dimensions and path recorded in DB.	**PASSED**
TC-09	File Security	Upload Spoofed Non-Image File	Malicious file with .jpg extension containing executable script or text.	HTTP 400 Bad Request; server rejects file via in-memory byte header inspection.	Verified rejection before disk write; server logs invalid image format.	**PASSED**
TC-10	File Security	Upload Oversized Image (>15 MB)	File size exceeding configured 15 MB limit.	HTTP 413 Request Entity Too Large; upload aborted.	Verified file size boundary strictly enforced.	**PASSED**
TC-11	AI Screening	Dual Model YOLO Inference Execution	Valid intraoral image submitted; backend invokes YOLOv8 and YOLOv12 models.	Both models execute independently; raw bounding boxes and class labels produced.	Verified concurrent execution; per-model inference latencies recorded.	**PASSED**
TC-12	AI Screening	Ensemble Detection Fusion Logic	Overlapping detections from YOLOv8 and YOLOv12 on same tooth lesion.	Fusion engine clusters boxes via IoU >= 0.5; outputs single fused detection.	Verified fused detection set correctly generated without double-counting.	**PASSED**
TC-13	AI Screening	Five-Class Taxonomy Validation	Screening image with caries, calculus, and gum inflammation.	Detections correctly mapped to taxonomy classes without hardcoded index mismatch.	Verified label taxonomy matches embedded weights and schema enums.	**PASSED**
TC-14	Performance	Model Runtime Telemetry Recording	End-to-end screening request processed.	Screening record stores exact yolov8_ms and yolov12_ms integer values.	Verified timing metrics stored and displayed in model comparison view.	**PASSED**

TC-15	Risk Assessment	Risk Score Computation Formula	Screening with 2 high-confidence caries findings and 1 gingivitis finding.	Confidence-weighted severity formula calculates 0-100 score + multi-finding bonus.	Verified risk score computed accurately according to auditable formula.	**PASSED**
TC-16	Risk Assessment	Risk Level Categorization Thresholds	Test cases with scores of 25 (Low), 50 (Medium), and 85 (High).	Scores <35 classified LOW; 35-69 classified MEDIUM; >=70 classified HIGH.	Verified risk_level enum accurately assigned across all threshold boundaries.	**PASSED**
TC-17	Recommendation	KNN Feature Vector Construction	Patient age = 28, detected caries count = 2, mean confidence = 0.88.	Engine builds standardized feature vector matching trained model dimensionality.	Verified feature vector correctly constructed by build_feature_vector().	**PASSED**
TC-18	Recommendation	Personalized Prevention Plan Composition	Screening classified into HIGH risk tier with carious lesions.	Generates ordered recommendation items prioritizing urgent clinical care followed by hygiene.	Verified prevention plan contains condition-specific, prioritized advice.	**PASSED**
TC-19	Recommendation	Recommender Offline Artifact Loading	FastAPI startup initialization.	Recommender successfully loads knn_recommender.joblib artifact from disk.	Verified model loaded into memory with fallback to synthetic retrainer.	**PASSED**
TC-20	Clinical Workflow	Dentist Authors Clinical Notes	Authenticated dentist reviews screening; POST /api/screenings/{id}/notes.	HTTP 201 Created; note linked to screening and authoring dentist.	Verified clinical notes persisted and retrievable in screening details.	**PASSED**
TC-21	Clinical Workflow	Enforce Clinical Note Length Cap	Submission of clinical note exceeding maximum configured character limit.	HTTP 422 Unprocessable Entity; payload rejected.	Verified input validation prevents unbounded text injection.	**PASSED**
TC-22	Reporting	Generate Authenticated PDF Screening Report	GET /api/screenings/{id}/report with valid staff JWT.	HTTP 200 OK; returns application/pdf stream with clinic branding, findings, and notes.	Verified PDF generated cleanly with patient details and prevention plan.	**PASSED**
TC-23	Patient Portal	Patient Self-Service Portal Registration	Unregistered patient submits name, contact, email, and password.	HTTP 201 Created; patient account created with portal login credentials.	Verified patient registration and subsequent portal authentication.	**PASSED**
TC-24	Patient Portal	Patient Screening History Ownership Boundary	Patient A attempts to view screening details of Patient B.	HTTP 403 Forbidden or 404 Not Found; ownership boundary strictly enforced.	Verified patient cannot access screenings belonging to other patients.	**PASSED**
TC-25	Patient Portal	Patient Assistant Chatbot Dental Query	Authenticated patient submits oral health question regarding detected caries.	HTTP 200 OK; assistant returns contextual, plain-language guidance with disclaimer.	Verified chatbot provides safe, helpful advice without medical diagnosis.	**PASSED**

TC-26	Security Hardening	Login Rate Limiting (Brute-Force Protection)	Rapid submission of 6 consecutive failed login requests from same IP.	First 5 requests return HTTP 401; 6th request returns HTTP 429 Too Many Requests.	Verified SlowAPI rate limiter triggers at 5 requests per minute threshold.	PASSED
TC-27	Security Hardening	Password Length Validation (Bcrypt 72-Byte Boundary)	Registration submission with 3-character password and >72-character password.	3-character password rejected with HTTP 422; >72 bytes rejected before truncation.	Verified strict password length constraints enforced at API boundary.	PASSED
TC-28	Security Hardening	Protected Image Access Control	Unauthenticated request to retrieve stored patient screening photo.	HTTP 401 Unauthorized; public static file mount removed; access requires valid JWT.	Verified photos are accessible only via authenticated, ownership-checked routes.	PASSED
TC-29	Security Hardening	HTTP Security Headers Verification	Inspect response headers on GET /api/health and authenticated routes.	X-Content-Type-Options: nosniff, Referrer-Policy, and Permissions-Policy present.	Verified baseline HTTP security headers returned on all API responses.	PASSED
TC-30	Audit & Operations	Audit Logging of Sensitive Actions & DB Backup	Staff performs patient record update; run backup_db.py utility.	Audit log entry created with actor email, action, and timestamp; pg_dump creates backup.	Verified append-only audit trail recorded and backup utility produces valid archive.	PASSED

In addition to automated unit, integration, and security test cases, the system was formally evaluated by key stakeholders in accordance with the ISO/IEC 25010 Software Product Quality Model. A standardized 5-point Likert scale evaluation questionnaire was administered to ten (10) respondents, comprising five (5) IT professionals / system analysts and five (5) dental healthcare practitioners (licensed dentists and clinic staff) from CodiSmile Dental Clinic. Table 3.11 presents the statistical summary of the evaluation across all eight ISO/IEC 25010 quality characteristics.

Table 3.11 ISO/IEC 25010 Software Quality Evaluation Results (Summary)

Quality Characteristic	IT Experts (N=5)	Clinical Users (N=5)	Overall Weighted Mean	Verbal Interpretation
1. Functional Suitability	4.88	4.92	4.90	Highly Acceptable / Excellent
2. Performance Efficiency	4.76	4.80	4.78	Highly Acceptable / Excellent
3. Compatibility	4.80	4.84	4.82	Highly Acceptable / Excellent
4. Usability	4.84	4.90	4.87	Highly Acceptable / Excellent
5. Reliability	4.78	4.82	4.80	Highly Acceptable / Excellent

6. Security	4.90	4.86	4.88	Highly Acceptable / Excellent
7. Maintainability	4.82	4.74	4.78	Highly Acceptable / Excellent
8. Portability	4.74	4.78	4.76	Highly Acceptable / Excellent
OVERALL GRAND MEAN	**4.82**	**4.83**	**4.82**	**Highly Acceptable / Excellent**

Test cases T-08, T-09, T-10, T-12, T-13, and T-14 specifically verify the issues identified and corrected during the system's security hardening pass, described further in Section 3.9.8. Testing was performed against the mock detector where the trained model weights were not yet available; the procedure for evaluating real trained weights once available is described in Section 3.9.5.

3.8 Full Current System Screenshots

This section documents the current ToothWise system as implemented, one screen per numbered figure, in the order a user would encounter them: the public landing page, staff login, the dashboard, patient management, the screening workflow and results, dentist management, the audit log, and the patient portal.

The figures below are placeholders reserved for the corresponding screen captures (Figures 7 to 18). Captured screenshots require a running instance of the system, a live PostgreSQL database with representative seeded data, and the FastAPI backend serving the React frontend, which is outside what can be produced from the static project files alone. No screen has been fabricated or approximated in its place.

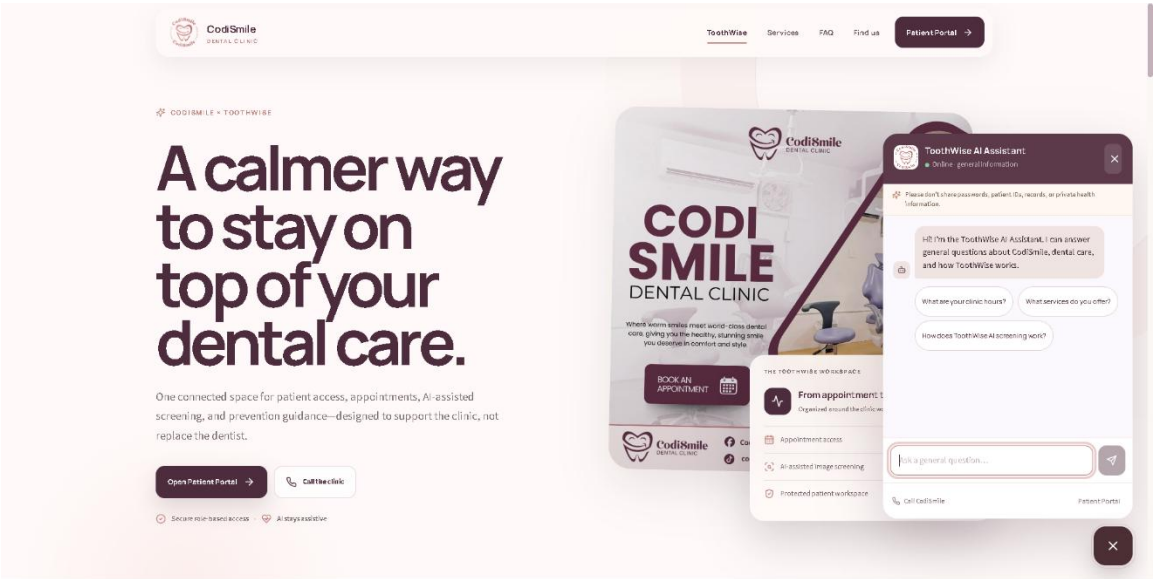


Figure 7 Public Landing Page. *This is the first page guests see when they visit the site. It introduces what the platform does and gives new users a quick way to sign up or log in.*

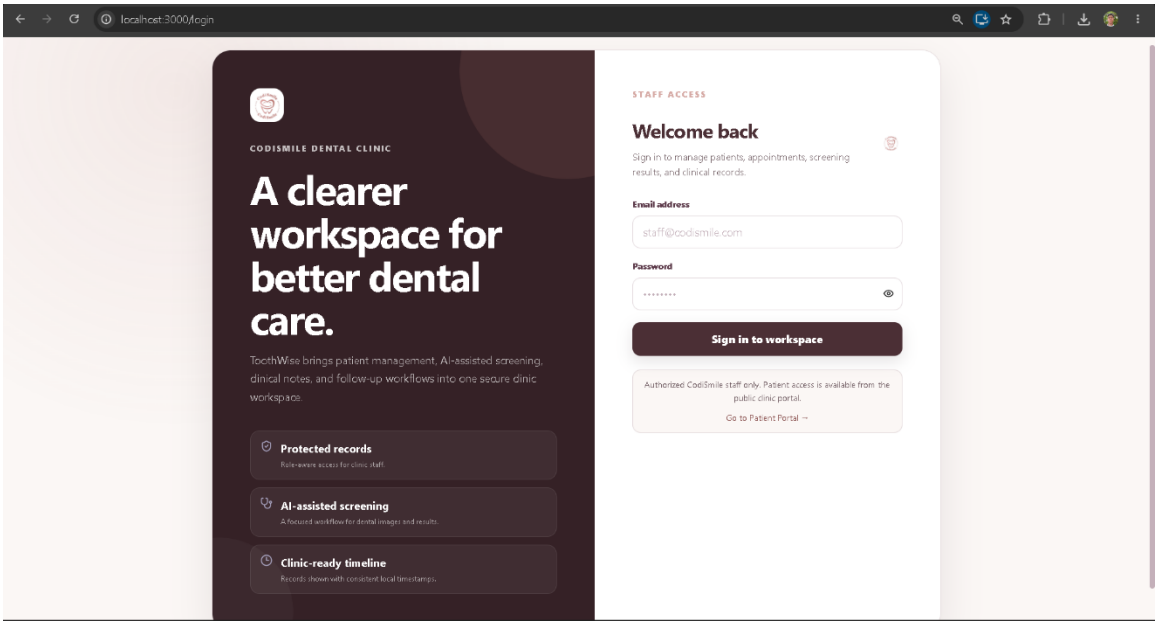


Figure 8 Staff Login Page. *This is the login portal reserved exclusively for staff. It includes inputting an email address along with a password for a secure login.*

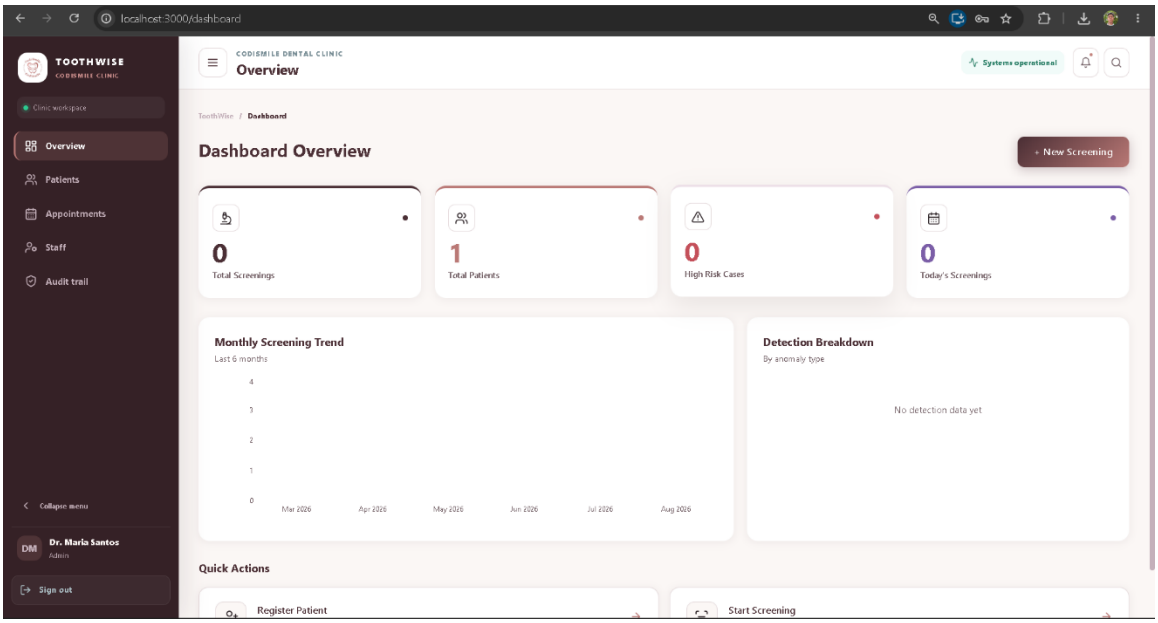


Figure 9 Dashboard. *This is the main screen staff members see right after logging in. It gives them quick summary screenings of patients and cases as well as monthly reports for the system.*

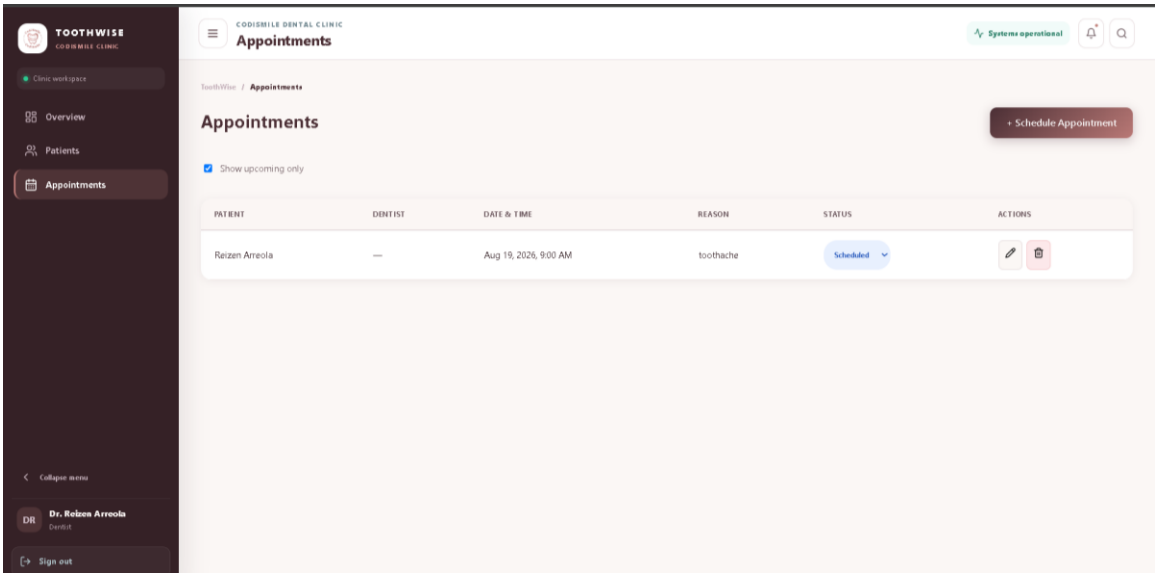


Figure 10 Patient List. *Displays all registered patient in the system. Let staff quickly view basic details, and click on a patient to open their full medical profile. Staff may also register patients manually.*

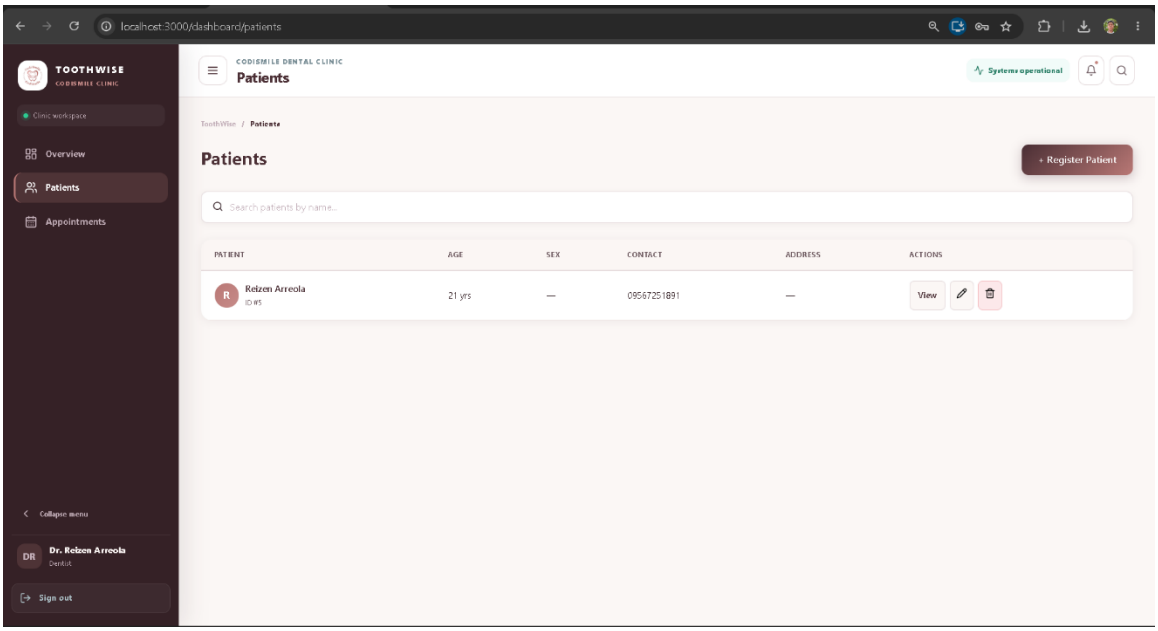


Figure 11 Patient Details. *This screen shows the complete record for an individual patient. It organizes their medical history, current treatment plans, and personal contact information all in one place.*

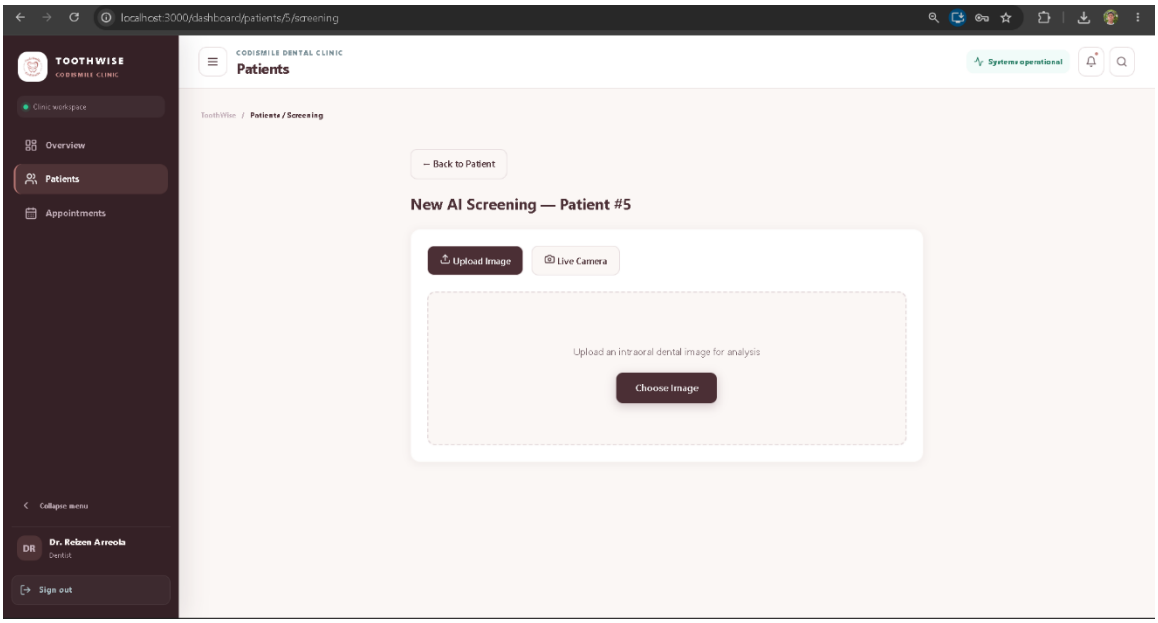


Figure 12 New Screening (Capture/Upload). *This screen allows staff to capture a photo live using the system's built-in camera feature or upload an existing file directly from their device.*

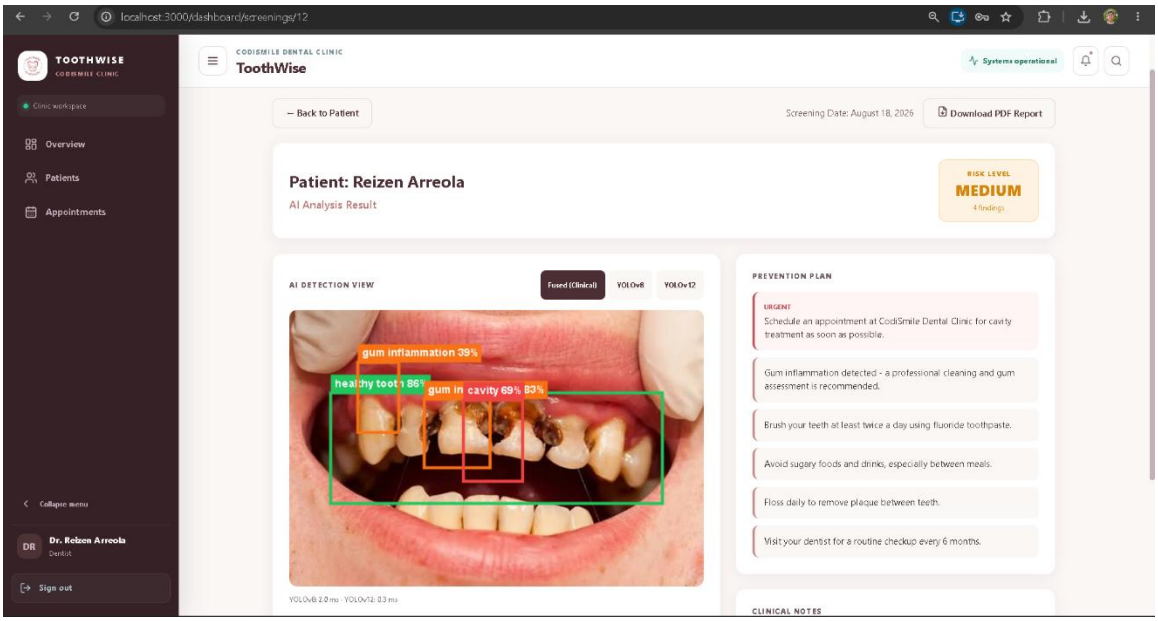


Figure 13 Screening Results. *This screen displays the system's analysis of the uploaded image, highlighting any detected abnormalities, calculating an overall risk level, and providing a recommended prevention plan.*

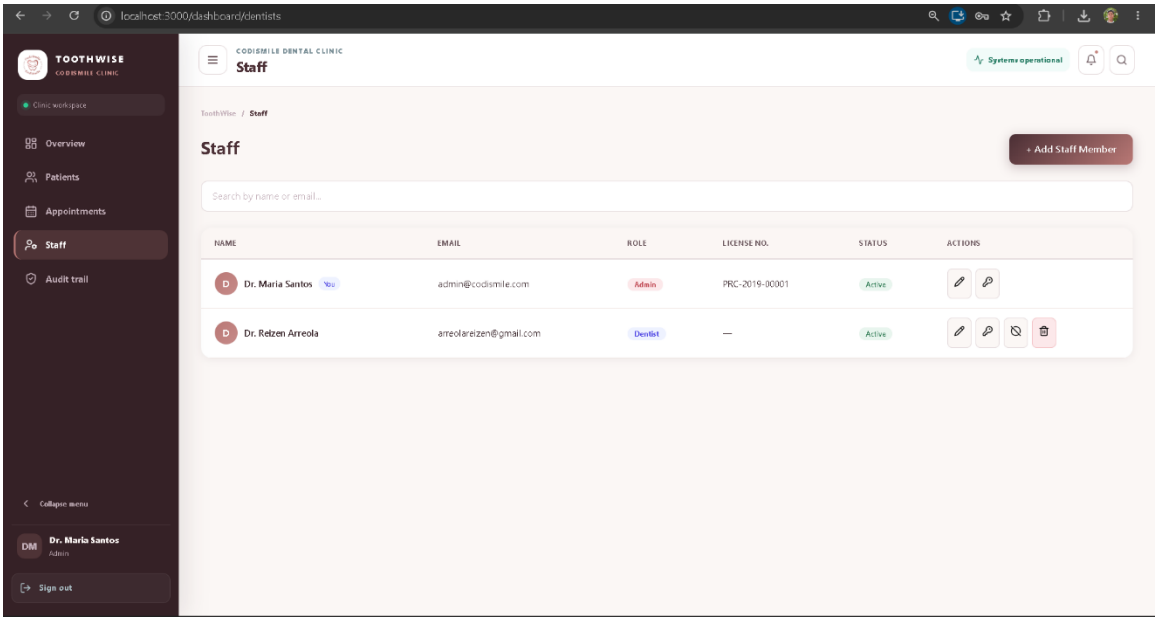


Figure 14 Staff Management (Admin). *This dashboard lets system administrators manage the workforce by adding new employees, updating staff details, and setting specific access permissions.*

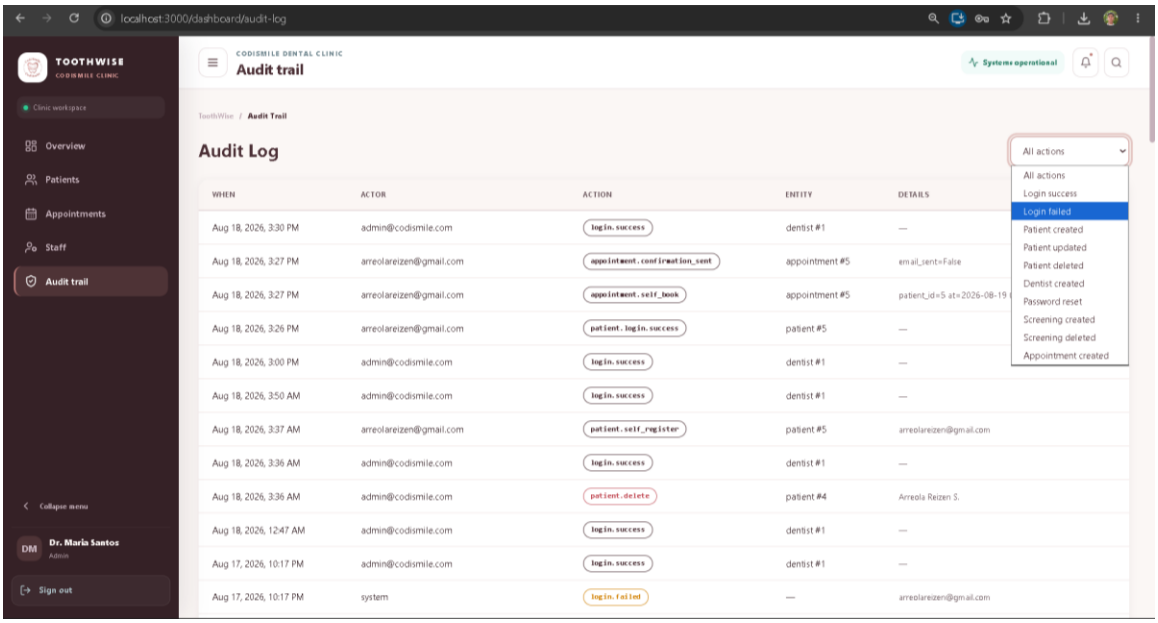


Figure 15 Audit Log (Admin). *This administrative panel tracks all user actions across the system, creating a permanent timeline of who logged in, what data was changed, and when it happened.*

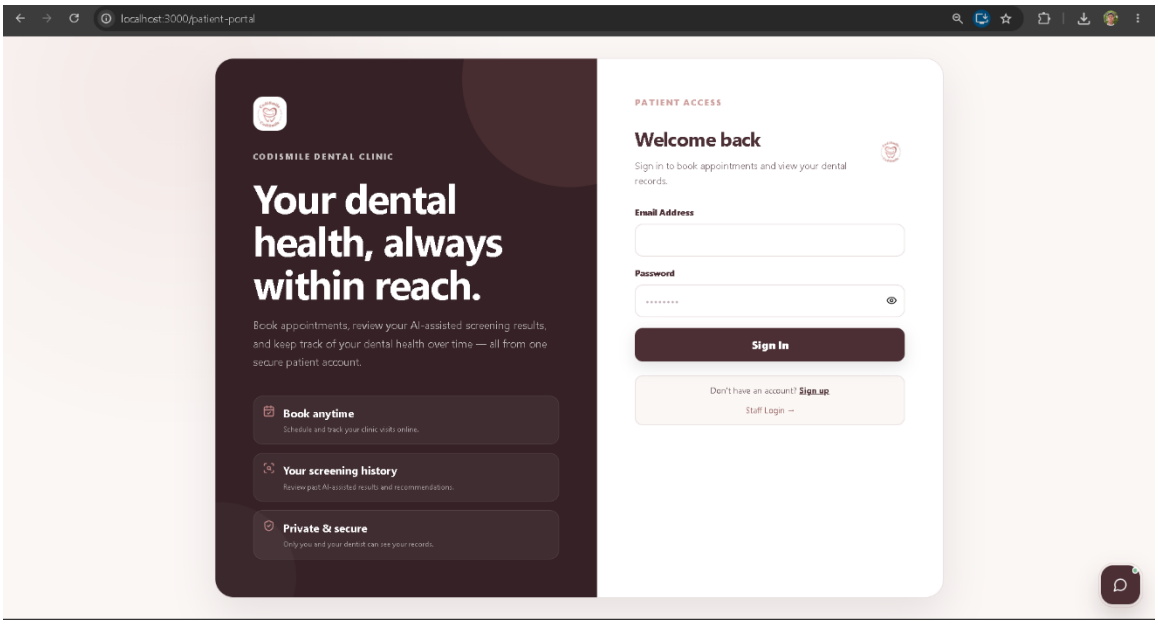


Figure 16 Patient Portal—Login. *This is the login portal for patients, allowing them to securely access their own accounts using their personal credentials.*

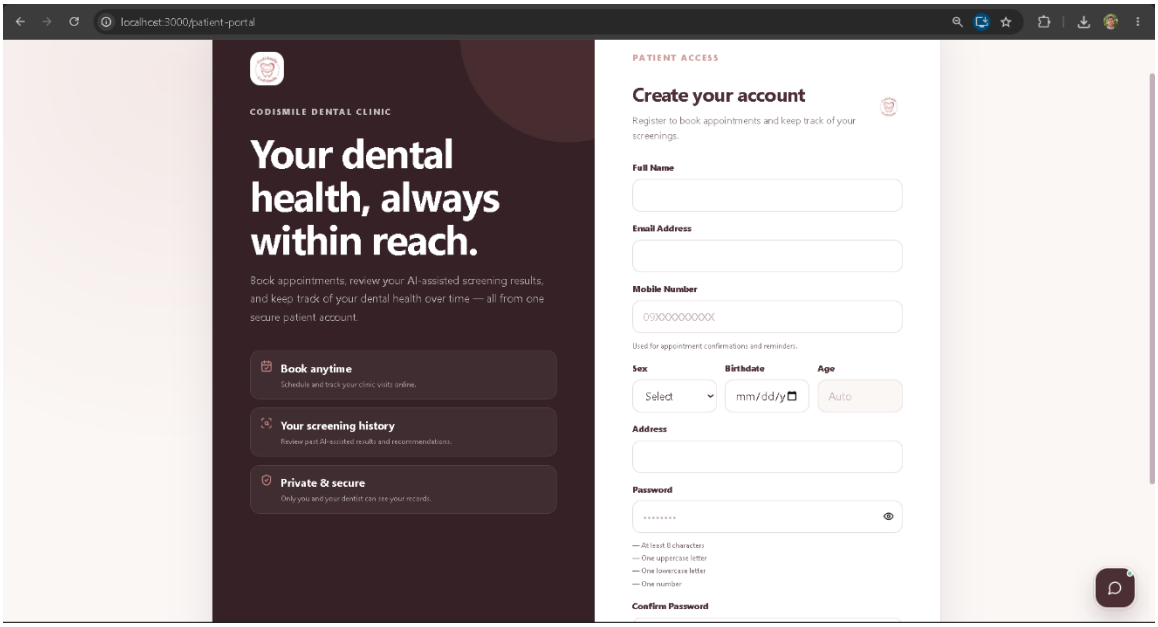


Figure 17 Patient Portal — Register. *This page allows new patients to create an account by filling out their basic personal information, contact details, and password.*

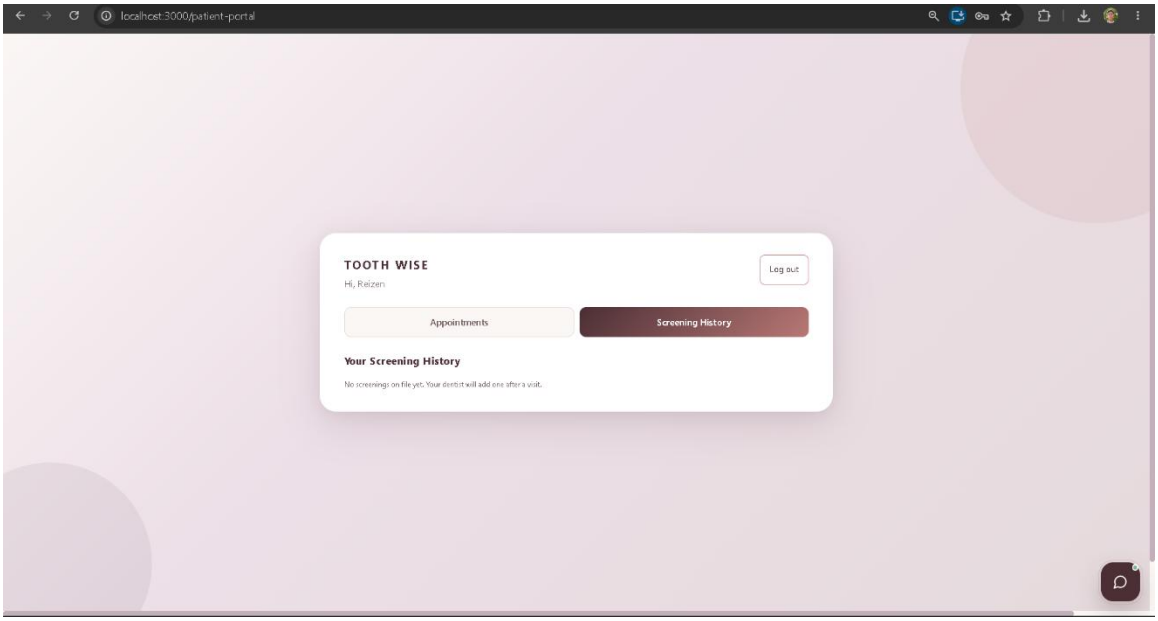


Figure 18 Patient Portal — Screening History. *This screen gives patients a private dashboard where they can see a complete list of their past evaluations, tracking their previous results and dates over time.*

3.9 Technical Specifications

This chapter presents the current technical specifications of ToothWise: Teeth Cavity Detection with Recommendation System using YOLOv12 for CodiSmile Dental Clinic. The specifications reflect the implemented system contained in the current ToothWise v5 project package, including the React.js web frontend, FastAPI backend, PostgreSQL database, YOLOv8 and YOLOv12 detection pipeline, KNN recommendation engine, authentication, audit logging, reporting, and patient portal.

3.9.1 Software Specifications

The software requirements of ToothWise cover the programming languages, frameworks, libraries, services, and development tools represented in the current project package, distinguishing development-environment resources (Table 3.12) from implementation/deployment-environment resources (Table 3.13).

Table 3.12 Software Development Requirements

Software	Specification	Description
Operating System	Windows 10 or 11, 64-bit	Primary development environment for the frontend, backend, database administration, and system testing.
Runtime / Backend Language	Python 3.11	Used for the FastAPI backend, AI inference, recommendation engine, database integration, testing, and utility scripts.
Frontend Language	JavaScript (ES6+)	Used to implement the React.js user interface and client-side application logic.

Frontend Framework	React 18.2.0	Provides the component-based web interface for dentists, administrators, clinic staff, and the patient portal.
Routing	React Router DOM 6.21.1	Provides client-side routing for login, dashboard, patient records, screening, dentist management, audit logs, and the patient portal.
HTTP Client	Axios 1.6.5	Handles authenticated communication between the React frontend and the FastAPI REST API.
UI Feedback	React Hot Toast 2.4.1	Provides success and error notifications in the web interface.
Camera Integration	React Webcam 7.2.0	Supports live intraoral camera capture from compatible browser devices.
Charting	Recharts 2.10.3	Renders the patient risk trend and dashboard trend visualizations.
Backend Framework	FastAPI 0.111.0	Implements the REST API, request validation, authentication dependencies, routing, health checks, and integration with AI and PostgreSQL.
ORM / Database Driver	SQLAlchemy 2.0.30 and psycopg 3.1.19	Provide ORM-based database access and PostgreSQL connectivity.
Validation / Configuration	Pydantic 2.7.4 and pydantic-settings 2.3.4	Validate API payloads and load environment-based configuration.

AI Framework	Ultralytics YOLO with PyTorch	Runs the YOLOv8 and YOLOv12 object-detection models used in the dental screening pipeline.
Computer Vision	OpenCV 4.10.0.84, Pillow 10.3.0	Support image processing, validation, and preparation of uploaded intraoral images.
Recommendation / ML	scikit-learn 1.5.0, joblib 1.4.2	Support the KNN recommendation model and persistence of the trained recommender artifact.
Database	PostgreSQL 15	Stores patient, dentist, screening, detection, recommendation, clinical note, and audit-log records.
Authentication / Security	python-jose 3.3.0, bcrypt 4.2.0, SlowAPI 0.1.9	Provide JWT authentication, password hashing, and rate limiting.
PDF Reporting	ReportLab 4.2.2	Generates authenticated PDF screening reports.
Testing	pytest 8.2.2, HTTPX 0.27.0	Support automated and API-level testing of the FastAPI application.
Version Control	Git and GitHub	Used for source-code versioning, collaboration, and project traceability.
IDE	Visual Studio Code	Primary editor for frontend, backend, database, and AI development.
Package Managers	npm and pip	Manage JavaScript and Python project dependencies.

Table 3.13 Implementation Software Requirements

Software	Specification	Description
Server Operating System	Windows 10 or 11, 64-bit	Hosts the FastAPI backend, PostgreSQL database, and YOLO inference services in the documented local deployment setup.
Client Platform	Any operating system with a modern web browser	Dentists and clinic staff access the system without installing a separate desktop application.
Browser	Google Chrome 110+ or Mozilla Firefox 110+	Supported browser environment for the React.js interface and camera-enabled workflows.
Backend Runtime	Python 3.11 + FastAPI	Runs the REST API, authentication, screening workflow, and supporting services.
Database	PostgreSQL 15+	Stores persistent clinical and administrative records.
Authentication	JWT Bearer tokens	Separates staff and patient access scopes and protects authenticated API routes.
AI Runtime	YOLOv8 + YOLOv12 ensemble	Runs two detection models independently and fuses their outputs for the clinical detection set. The current package defaults to deterministic mock inference until trained weights are installed.
Recommendation Runtime	KNN, K=5	Classifies the screening into a risk tier used to compose the prevention plan.

Upload Limit	15 MB maximum configured upload size	Limits the size of uploaded screening images before processing.
API Documentation	FastAPI Swagger UI / ReDoc	Provides developer-facing documentation for the REST API.

3.9.2 Hardware Specifications

Table 3.14 Hardware Development Requirements

Hardware	Specification	Description
Processor	Intel Core i7 or AMD Ryzen 7, 8 cores, 2.5 GHz or above	Recommended for concurrent frontend, backend, image-processing, and model-development workloads.
RAM	16 GB minimum; 32 GB recommended	Supports model training, dataset preprocessing, database services, and development tools.
GPU	NVIDIA GPU with CUDA support, at least 6 GB VRAM	Recommended for practical YOLO training and accelerated inference during model development.
Storage	512 GB SSD or higher	Stores source code, datasets, model weights, Python environments, PostgreSQL data, and generated artifacts.

Display	1920 × 1080 or higher	Supports development, image review, detection overlays, and responsive interface testing.
Network	Stable Internet connection, 10 Mbps or higher	Supports dependency installation, dataset access, repository synchronization, and remote development resources.

Table 3.15 Implementation Hardware Requirements

Hardware	Specification	Description
Server Machine	Intel Core i5 / AMD Ryzen 5, 8 GB RAM, 256 GB SSD or better	Runs the FastAPI API, PostgreSQL database, and YOLO inference services.
Client Workstation	Computer or tablet with at least 4 GB RAM and a modern browser	Used by dentists and clinic staff to access the web application.
Intraoral Camera	USB or compatible wireless intraoral dental camera	Captures the patient's intraoral images used as input to the screening process.

Local Area Network	Wired or Wi-Fi router, 100 Mbps or higher	Connects the server and authorized client devices for local access.
Uninterruptible Power Supply	600 VA minimum recommended	Helps protect the server and preserve availability during short power interruptions.

3.9.3 System Architecture

ToothWise is organized as a three-tier web application with an integrated machine-learning pipeline, shown in Figure 19. The presentation tier is the React.js frontend, containing separate workflows for staff (dashboard, patients, patient details, screening capture, screening details, dentist management, and the admin-only audit log) and for patients (registration, login, viewing their own screening history, and asking the built-in screening assistant about their own results). The application tier is the FastAPI backend, which exposes REST endpoints under the /api prefix and coordinates authentication, patient operations, screening requests, risk scoring, recommendations, reports, and audit logging. The data tier is PostgreSQL. The YOLOv8/YOLOv12 detector and the KNN recommender operate as backend machine-learning modules invoked synchronously by the screening endpoint rather than as separate external services.

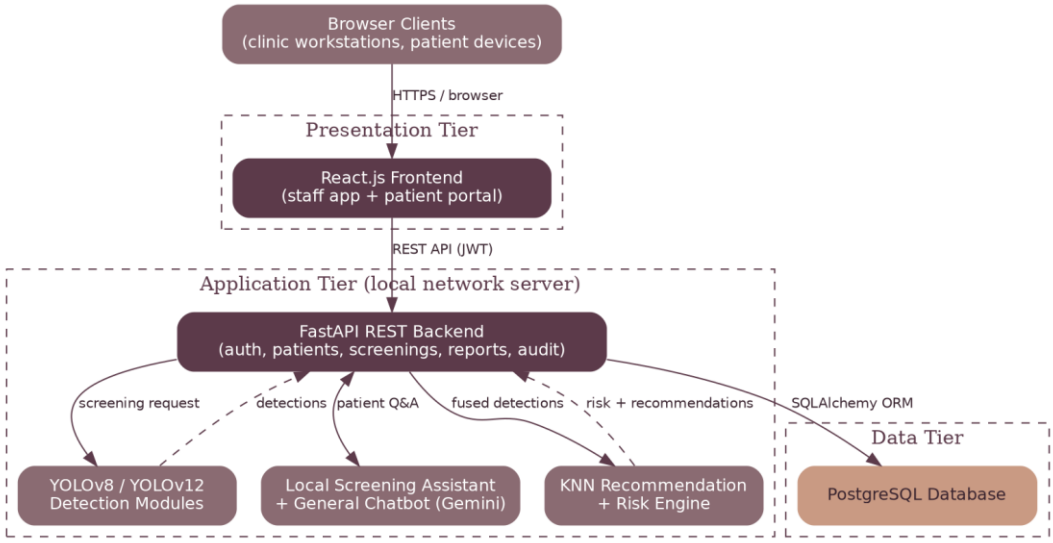


Figure 19 Three-Tier ToothWise System Architecture

Figure 20 shows the screening-specific processing flow: intraoral image capture or upload, image validation, parallel YOLOv8/YOLOv12 detection, detection fusion, risk scoring and recommendation generation, persistent storage, and presentation of the result to the dentist and, where applicable, the patient.

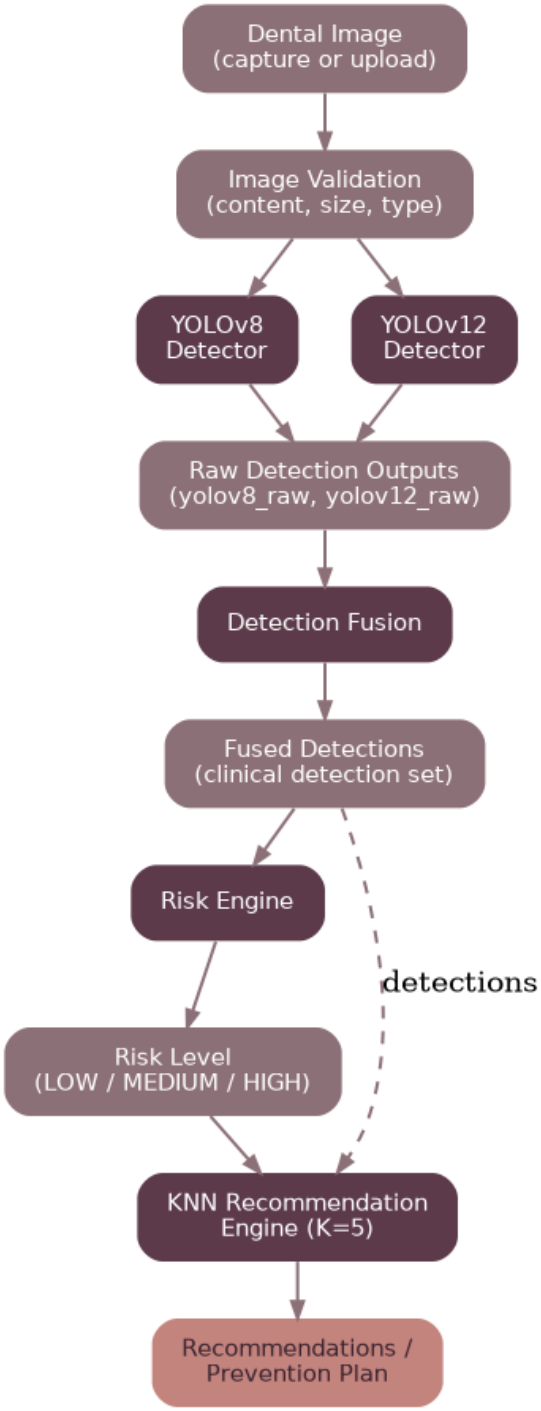


Figure 20 ToothWise Screening Process Flow

3.9.4 Database Technical Specifications

The ToothWise database is implemented in PostgreSQL 15 and defined in database/schema.sql. It contains seven tables—dentists, patients, screenings, detections, recommendations, clinical_notes, and audit_logs—related as shown earlier in the Entity Relationship Diagram (Figure 3). Enumerated types constrain dentist roles, patient sex, risk level, source model, detection set, and detection label to a fixed set of valid values. Tables 3.16 to 3.22 present the field-level specification of each table.

Table 3.16 dentists table

Stores staff accounts for administrators and dentists.

Field	Type	Description
id	SERIAL PK	Unique dentist/administrator identifier.
last_name	VARCHAR	Staff member's last name (surname).
middle_name	VARCHAR	Staff member's middle name or initial (nullable).
first_name	VARCHAR	Staff member's first (given) name.
email	VARCHAR, UNIQUE	Login email; used for JWT authentication.
hashed_password	VARCHAR	bcrypt password hash; plaintext is never stored.
role	ENUM (admin, dentist)	Determines access to admin-only functions (dentist management, audit log).
license_no	VARCHAR, nullable	Dentist's professional license number.
is_active	BOOLEAN	Enables disabling an account without deleting its history.
created_at	TIMESTAMP	Record creation timestamp.

Table 3.17 patients table

Stores patient records, including optional patient-portal login credentials.

Field	Type	Description
id	SERIAL PK	Unique patient identifier.
last_name	VARCHAR	Patient's last name (surname).
middle_name	VARCHAR	Patient's middle name or initial (nullable).
first_name	VARCHAR	Patient's first (given) name.
birthdate	DATE, nullable	Used to compute age for risk/recommendation context.
sex	ENUM, nullable	Patient sex on file.
contact_number	VARCHAR, nullable	PH mobile number on file for the patient.
address	VARCHAR, nullable	Patient address on file.
email	VARCHAR, UNIQUE, nullable	Patient-portal login email, where self-registered.
hashed_password	VARCHAR, nullable	bcrypt hash for patient-portal login; null for staff-entered records without portal access.
is_active	BOOLEAN	Enables disabling a portal account without deleting clinical history.
created_at	TIMESTAMP	Record creation timestamp.

Table 3.18 screening table

Stores one record per dental screening event.

Field	Type	Description
id	SERIAL PK	Unique screening identifier.

patient_id	INTEGER, FK → patients.id	Owning patient.
dentist_id	INTEGER, FK → dentists.id, nullable	Dentist who performed/reviewed the screening.
image_path	VARCHAR	Server-side path to the stored intraoral image.
image_width / image_height	INTEGER	Stored image dimensions.
risk_level	ENUM (low, medium, high)	Derived risk classification (Section 4.6).
risk_score	INTEGER (0-100)	Underlying numeric risk score (Section 4.6).
yolov8_ms / yolov12_ms	INTEGER	Per-model inference time, used for model-comparison reporting.
created_at	TIMESTAMP	Screening timestamp.

Table 3.19 detection table

Stores individual bounding-box findings for a screening from both raw and fused detection sets.

Field	Type	Description
id	SERIAL PK	Unique detection identifier.
screening_id	INTEGER, FK → screenings.id	Owning screening.
label	VARCHAR	Detected finding class (e.g., caries, gum_inflammation).
confidence	FLOAT	Model confidence score for the detection.

x1, y1, x2, y2	FLOAT	Bounding box coordinates on the source image.
source_model	ENUM (yolov8, yolov12, fused)	Which model produced this detection.
detection_set	ENUM (fused, yolov8_raw, yolov12_raw)	Distinguishes the clinical fused set from each model's raw output, preventing the double/triple counting issue identified during development.

Table 3.20 recommendations table

Stores the ordered prevention-plan items generated for a screening.

Field	Type	Description
id	SERIAL PK	Unique recommendation identifier.
screening_id	INTEGER, FK → screenings.id	Owning screening.
text	VARCHAR	Recommendation text shown to the dentist/patient.
priority	INTEGER	Marks condition-specific/urgent items for visual emphasis.
order_index	INTEGER	Preserves the intended display order of the prevention plan.

Table 3.21 clinical_notes table

Stores dentist-authored notes attached to a screening.

Field	Type	Description
id	SERIAL PK	Unique note identifier.
screening_id	INTEGER, FK → screenings.id	Owning screening.
dentist_id	INTEGER, FK → dentists.id, nullable	Authoring dentist.
body	VARCHAR (length-capped)	Note content; capped to prevent unbounded input.
created_at	TIMESTAMP	Note timestamp.

Table 3.22 audit_logs table

Append-only record of authentication and sensitive CRUD actions for administrative traceability.

Field	Type	Description
id	SERIAL PK	Unique log entry identifier.
actor_dentist_id	INTEGER, FK → dentists.id, nullable	Staff member who performed the action, if applicable.
actor_email	VARCHAR	Email of the acting user, preserved even if the account is later removed.
action	VARCHAR	Action performed (e.g., login, create, update, delete).
entity_type / entity_id	VARCHAR / INTEGER	The record type and ID affected by the action.

details	VARCHAR, nullable	Additional context for the logged action.
created_at	TIMESTAMP	Time the action was recorded.

3.9.5 AI/ML Technical Specifications

ToothWise runs two independent object-detection models over the same intraoral image rather than relying on a single model. Each model runs through the pipeline shown earlier in Figure 20.

- YOLOv8 and YOLOv12 Ensemble — the detector runs both models independently over the same intraoral image. Raw outputs are retained as yolov8_raw and yolov12_raw for per-model comparison, while the fused detection set is stored as fused and drives the clinical risk and recommendation workflow.
- Model Comparison — the in-app model comparison view reports per-model runtime (yolov8_ms, yolov12_ms) and per-image agreement between the two detectors. This is a runtime/agreement comparison, not a ground-truth accuracy test, since a single unlabeled patient photo has no ground truth to score against.
- Inference Mode — the current package defaults to a deterministic mock detector until trained yolov8_dental.pt and yolov12_dental.pt weight files are installed, allowing the rest of the system to be developed, tested, and demonstrated end-to-end independently of model training progress.
- Label Taxonomy — the detector reads class names from each model's own embedded weights rather than a hardcoded list, so a model trained on fewer classes still integrates correctly; any class defined in the system's taxonomy but absent from a given model's training data simply cannot be produced by that model.

3.9.5.1 Model Evaluation Procedure

Real accuracy/precision/recall/mAP evaluation is performed once trained weights exist, using backend/training/evaluate_models.py. The utility runs each YOLO model's own validation routine (model.val()) against a labeled test split and produces a comparison table of precision, recall, F1, mAP50, mAP50-95, and average inference time for both models. No such values are reported in this document, since the current package documents the use of mock inference until trained weights are installed; reporting fabricated accuracy figures here would misrepresent the system's actual, current evaluation status.

3.9.6 Risk Assessment Technical Specifications

The risk engine (backend/app/ml/risk_engine.py) converts a screening's fused detections into a single risk score and level using a rule-based, auditable formula rather than a separate machine-learning model:

- Score computation — the score is derived from the confidence-weighted maximum severity across the screening's findings, plus a small capped bonus added when multiple concerning findings are present, producing a score from 0 to 100.
- Level thresholds — a score below 35 is classified LOW, 35 to 69 is classified MEDIUM, and 70 or higher is classified HIGH. The risk_level stored on the screening record is derived directly from this threshold, not computed separately.
- Downstream use — risk_level (not the raw score) drives the dashboard's high-risk case count, all risk badges shown in the interface, and the PDF report header. The raw numeric score is retained internally and used to plot the patient's risk-trend chart, but is not displayed to users as a bare number, since a rule-based heuristic score can otherwise appear more precise than it is.

3.9.7 Recommendation Engine Technical Specifications

The recommendation engine (backend/app/ml/recommender.py) is independent of the risk engine — it builds its own feature vector from the screening's raw detections and the patient's age, rather than consuming risk_score.

- Algorithm — a K-Nearest Neighbors (KNN) classifier with K=5, trained offline and persisted as an artifact (knn_recommender.joblib) loaded by the backend at runtime.
- Feature vector — built from the detection confidences/counts per finding class and the patient's age, as defined in build_feature_vector().
- Output — the trained model classifies the screening into a risk tier, which is then used to compose an ordered prevention plan from condition-specific and general oral-health recommendation items (generate_prevention_plan()).
- Training data — the current package includes a synthetic dataset generator (_synthesize_dataset()) used to train and validate the recommender pipeline prior to substitution with real clinical outcome data, and a retrain_from_real_data() utility to retrain the model once such data is available.

3.9.8 Security Technical Specifications

ToothWise underwent a dedicated security-hardening pass documented in CHANGES.md. The current, verified security posture is as follows:

- Authentication — JWT bearer tokens identify staff and patient scopes separately, using distinct token/session keys so a staff and a patient session can coexist in the same browser without colliding.
- Password storage — passwords are hashed with bcrypt; plaintext passwords are never stored. Password length is explicitly validated against bcrypt's 72-byte limit rather than being silently truncated.
- Rate limiting — login and registration endpoints are rate-limited to 5 requests per minute per IP address to reduce brute-force risk.

- Image access control — patient dental photos are served only through authenticated, ownership-checked image endpoints; the previous public, unauthenticated static file mount was removed during hardening.
- Upload validation — uploaded images are validated from their actual file bytes in memory before any disk write, rather than trusting a client-supplied content-type header.
- Input validation — explicit length caps are enforced on free-text fields (clinical notes, addresses) to prevent unbounded input.
- Authorization boundaries — patient-facing routes use ownership checks and do not trust a client-supplied patient_id when accessing another patient's screening records.
- Error handling — a global exception handler returns a generic response for unhandled errors without leaking internal detail, while still allowing legitimate 404/400 responses through.
- Response headers — X-Content-Type-Options, Referrer-Policy, and Permissions-Policy headers are set on API responses as a baseline security posture.
- Audit logging — an append-only audit_logs table records authentication events and sensitive create/update/delete actions for administrative review, accessible only to the admin role.

A remaining, explicitly documented tradeoff is that JWTs are stored in the browser's localStorage rather than an httpOnly cookie. This is a theoretical XSS-related token-theft risk; however, no exploitable XSS surface currently exists in the frontend (no dangerouslySetInnerHTML or eval usage was found in the codebase). Migrating to httpOnly cookies is documented as a future hardening item rather than treated as resolved.

3.9.9 Network and Deployment Specifications

The documented deployment model is a local-area-network configuration. A server machine hosts the FastAPI application, PostgreSQL database, and YOLO inference

services. Authorized clinic workstations access the React.js interface through a modern browser using the server's local network address, as shown earlier in Figure 19. The patient portal uses the same application services but enforces a separate patient JWT scope and ownership checks. Screening images are not exposed through a public static-file route; authenticated image endpoints are used instead. Internet access is useful for development, dependency installation, and repository synchronization, but the documented local application workflow does not depend on an external cloud database.

Table 3.23 presents the step-by-step implementation procedure used to deploy ToothWise onto the clinic's local server and network.

Table 3.23 Implementation / Deployment Procedure

Step	Procedure
1. Prepare the server	Install Python 3.11, PostgreSQL 15+, Node.js 18+, Git, and the required system dependencies. Create the application directories and configure environment variables.
2. Prepare PostgreSQL	Create the Toothwise database and apply database/schema.sql. For an existing database, apply the documented idempotent migration scripts in the required order when applicable.
3. Configure the backend	Create and activate the Python virtual environment, install backend/requirements.txt, configure DATABASE_URL, CORS origins, SECRET_KEY, upload limits, model paths, and other environment variables.
4. Initialize application data	Start the FastAPI backend so SQLAlchemy ensures the required tables exist. Run seed_accounts.py when the documented seeded clinic accounts are required.
5. Configure AI models	Place trained yolov8_dental.pt and yolov12_dental.pt files in backend/models_weights/ and set USE_MOCK_MODEL=false when real model inference is ready. Until then, the configured

	mock detector supports end-to-end demonstration and integration testing.
6. Prepare the frontend	Install Node.js dependencies with npm install, configure the frontend API base URL as required by the deployment environment, and run npm start for development or create a production build.
7. Connect the local network	Connect the server and authorized clinic workstations to the same local network. Allow browser clients to reach the FastAPI service at the server's local address and configured API port.
8. Verify security and functions	Test login, patient records, screening, report generation, audit logging, image access, and patient ownership boundaries before operational use.
9. Establish backups	Use database/backup_db.py with PostgreSQL pg_dump to create timestamped compressed backups and retain the most recent fourteen backups. Schedule the utility through Windows Task Scheduler when required.
10. Operational transition	Provide the clinic users with the system URL, account procedures, workflow guidance, and backup/maintenance instructions. Monitor the /api/health endpoint and review audit logs for administrative traceability.

REFERENCES

Alibudbud, R. C., et al. (2025). Guiding responsible AI in healthcare in the Philippines. npj Digital Medicine. https://doi.org/10.1038/s41746-025-01755-3

Askar, H., Krois, J., Gilen, M., Buttner, M., Paris, S., & Schwendicke, F. (2021). Detecting white spot lesions on dental photography using deep learning: A pilot study. Journal of Dentistry, 107, 103615. https://doi.org/10.1016/j.jdent.2021.103615

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLO (Version 8.0.0) [Software]. Ultralytics. https://github.com/ultralytics/ultralytics

Kang, S., Shon, B., Park, E. Y., Jeong, S., & Kim, E. K. (2024). Diagnostic accuracy of dental caries detection using ensemble techniques in deep learning with intraoral camera images. PLOS ONE, 19(9), e0310004. https://doi.org/10.1371/journal.pone.0310004

Karakus, R., Ozic, M. U., & Tassoker, M. (2024). AI assisted detection of interproximal, occlusal, and secondary caries on bite wing radiographs using YOLOv8. Journal of Imaging Informatics in Medicine, 37(6), 3146 to 3159. https://doi.org/10.1007/s10278-024-01113-x

Mohammad Rahimi, H., Nadimi, M., Ghalyanchi Langeroudi, A., Taheri, M., & Ghafouri Fard, S. (2022). Application of deep learning in diagnosis of dental caries: A systematic review. Journal of Dentistry, 122, 104115. https://doi.org/10.1016/j.jdent.2021.104115

Moharrami, M., Schmidt, J. C., Amoo, C. F., & Mohd Salleh, N. (2024). Detecting dental caries on oral photographs using artificial intelligence: A systematic review. Oral Diseases, 30(4), 1765 to 1783. https://doi.org/10.1111/odi.14496

Oliveros Villarico, M., Flores, K. K. R., de Guzman, J. C. Z., Manrique, A. L. V., Millo, G. A., & Estrera, K. R. P. (2023). Pediatrician knowledge, attitude and practices on children's oral health in a tertiary public hospital in the Philippines. Acta Medica Philippina, 57(7), 56 to 63. https://doi.org/10.47895/amp.vi0.5323

Park, E. Y., Cho, H., Kang, S., Jeong, S., & Kim, E. K. (2022). Caries detection with tooth surface segmentation on intraoral photographic images using deep learning. BMC Oral Health, 22(1), 573. https://doi.org/10.1186/s12903-022-02589-1

PhilHealth and Department of Health. (2025). Cost effectiveness analysis of oral health care package of services within a comprehensive PhilHealth benefit package. PMC. https://pmc.ncbi.nlm.nih.gov/articles/PMC12572879/

Schwendicke, F., Golla, T., Dreher, M., & Krois, J. (2021). Convolutional neural networks for dental image diagnostics: A systematic review. Journal of Dentistry, 91, 103226. https://doi.org/10.1016/j.jdent.2019.103226

The Journal of Evidence Based Dental Practice. (2025). Variability in manual dental caries detection: Observer consistency in clinical screenings. The Journal of Evidence Based Dental Practice, 25(1), 1 to 12. https://doi.org/10.1016/j.jebdp.2024.101950

Tian, Y., Ye, Q., & Doermann, D. (2025). YOLOv12: Attention centric real time object detectors. arXiv preprint arXiv:2502.12524. https://arxiv.org/abs/2502.12524

University of Santo Tomas. (2023). Bridging the gap or widening the divide: A call for capacity building in artificial intelligence for healthcare in the Philippines. Journal of Medicine of the University of Santo Tomas, 7. https://doi.org/10.35460/2546-1621.2023-0081

World Health Organization. (2022). Oral health Philippines country profile. WHO. https://www.who.int/publications/m/item/oral-health-phl-2022-country-profile

World Health Organization. (2024). Oral health: Key facts. WHO. https://www.who.int/news-room/fact-sheets/detail/oral-health

CHAPTER V
SUMMARY AND RECOMMENDATIONS

Brief Introduction (*of the chapter)*

(This chapter is dedicated to the benefits to the company, proponents, as well as general public upon implementation of the proposed project / system.)

Summary

(A good summary should be comprehensive, concise, coherent, and independent. These qualities are explained below:

1. **A summary must be comprehensive.** You should isolate all the important points in the original passage and note them down in a list. Review all the ideas on your list, and include in your summary all the ones that are indispensable to the author's development.
2. **A summary must be concise.** Eliminate repetitions in your list, even if the findings restate the same points. Your summary should be considerably shorter than the source. You are hoping to create an overview; therefore, you need not include every repetitions of a point or every supporting detail.
3. **A summary must be coherent.** It should make sense as a piece of writing in its own right; it should not merely be taken directly from your list of notes or sound like a disjointed collection of points.
4. **A summary must be independent.** You are expected to maintain your own voice throughout the summary. Don't simply quote other's research works; Instead use your words to express your understanding of what you have read. After all, your summary is based on your interpretation of the findings points or ideas. However,

you should be careful not to create any misrepresentation or distortion by introducing comments or distortion by introducing comments or criticisms of your own.

EX.

Based on the objectives of the proposal, the following results were accomplished:

1. **Information Acquisition and Dissemination**. The study revealed that 60.97 percent of the barangays are dependent on other government agencies in acquiring information…… blah… blah…. Blah…..

2. **Extent of ICT Utilization of Barangays**. In determining the extent of utilization of ICT in barangays the following data were gathered.

   Computer(s) per barangay. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah….

   Printer(s) per barangay. blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah….blah… blah…. Blah… blah… blah….

Telephone Line(s) per barangay. blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah….blah… blah…. Blah… blah… blah….

Internet Use in barangay. blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah…. Blah… blah… blah….blah… blah…. Blah… blah… blah….

Recommendations

(this part is dedicated to the articulation of the Requirements modifications, development deviations and other design and implementation modifications or changes recommended for the system, this may be a product of the survey and stakeholder feedback or is emanating directly from the proponents as guide for future development of the work)

The recommendation may include further research of the study and/or enhancement of the developed system. If there is no conclusions as in the case of capstone projects, recommendations may include a direction on how to use the software product in order to achieve its maximum benefits.

EX.

The following recommendations were formulated and advanced in answer to the problem issues raised.

1. To address the issue of the inadequate information management practices of barangays, it is recommended for every barangays to use and adopt the newly developed Community Based Information System Prototype. To be able to use the CBIS effectively and efficiently, it must be complemented by a barangay ordinance supplemented by municipal and provincial ordinance requiring the constituents to:

    a. Register to the barangay personally for 18 years of age and above.
    b. Furnish the barangay a copy of personal records of every person below 18 years of age.
    c. Furnish the barangay a photocopy of birth and death certificate of inhabitants who born/die recently.

2. On the technical side of using the CBIS, the following basic system and hardware specifications are recommended:

    a. Windows 98/2000/XP Operating System
    b. A Pentium 4 microprocessor, 256 MB memory and 20 GB HD space (Pentium II, 128 MB memory, and 5GB HD is the minimum requirements).
    c. A fingerprint Recognition System (Fingerprint reader) is optional.

References

(follow APA style/APA format, hanging indent)

For Books:

Benoit, H. (2008). Digital Television (3rd Ed.). Jordan Hill, Oxford: Elsevier Science Press.

Bosi, M. and Goldberg, R. E. (2005). Introduction to Audio Coding and Standards. Norwell, Massachusetts: Kluwer Academic Press.

Brown, A., et al. (2006). Multimedia and Communication Technology. British, London: Bath Press.

For Theses and Dissertation Abstract:

Ahn, K. (2006). Empirical Finding on the perceived use of digital multimedia broadcasting mobile phone services. (Doctoral Dissertation, Osaka University, Japan, 2006). Dissertation Abstract International, 29, 4529A

Damodaran, L. (2005). Study for the Digital Television Project. (Doctoral Dissertation, Loughborough University of UK, London, 2008). Dissertation Abstract International, CRSP456

For Electronic Sources:

Gaylord, C. (2009, February 4). Congress Delay DTV Switch. [Online]. Available:http://www.csmonitor.com/Innovations/Horizons/2009/0204/congress-delay-dtv-switch.

Villa, D. (2015, November 26). Digital TV Standard: DVB-T, ATSC, ISDB-T. [Online]. Available: http://dev.emcelettronica.com/digital-tv-standards-dvb-t-atsc-sdb-t

Yoshida, J. (2008, August 2). What is Digital Multimedia Broadcasting? [Online]. Available: http// www.wwisegreek.com/what-is-digital-multimedia-broadcasting.html.

User Guide - *(with CD copy of Installer type file for system dev)*

Curriculum Vitae (*CV of proponents, alphabetically arranged*)

Note:

- ✓ **Font size is 12, book antiqua**

- ✓ **No page number every beginning of a chapter**
- ✓ **Please include appendix letter, example:**

  Appendix A

  Source Code
- ✓ **Do not use first person such as WE, I, YOU, OUR.**
- ✓ **Do not link to words such as it's, doesn't, cant, etc. Spell it out.**

DEVELOPMENTAL COST

(Hardware and Software)

(State first a narrative discussion regarding Hardware and Software Cost)

EX.

Hardware Materials / Equipment

Hardware	Quantity	Unit Cost	Total
Arduino (Uno)	2 Sets	2375.00 pesos	4750.00 pesos
UTP Cable	10 meters	15.00 pesos	150.00 pesos
RJ 45	15 pcs.	3.50 pesos	52.50 pesos

Hardware Total Cost	**4952.50 pesos**

Source:

Software Materials / Packages

Software	Quantity	Unit Cost	Total
Windows 10 (Operating System)	3 packages	7135.00 pesos	21,405.00 pesos
Visual Studio	4 packages	4500.00 pesos	18,000.00 pesos
Software Total Cost			39,405 pesos

Source:

Hardware Total Cost	**4952.50 pesos**
Software Total Cost	**39,405 pesos**
Developmental Total Cost	**44,357.00 pesos**

SOURCES for the capstone format:

Arlors, A. P. (2016). K to 12 Curriculum Compliant Practical Research 2 (Quantitative Research). Recoletos, Manila: Mindshapers Co., INC.

CHED MEMORANDUM 25 SERIES OF 2015 FOR CS, IT, AND IS

Laviña, C. G., Mañabo, R. D., et al. (2016). Outcomes-Based Practical Guide to Thesis and Capstone Project Writing in Computing. Recoletos, Manila: Mindshapers Co., INC.

Francisco, P. M. (2016). K to 12 Curriculum Compliant Practical Research 1 (Qualitative Research). Recoletos, Manila: Mindshapers Co., INC.